

**UNIFIED
MODELING
LANGUAGE™**



TOPIC 1

INTRODUZIONE

- Un **modello** è un'astrazione che cattura le **proprietà** salienti della realtà che si desidera rappresentare.
- **Idealizza una realtà complessa**, individuandone i tratti importanti e separandoli dai dettagli, facilitandone la comprensione.
- La mente umana compie un'attività continua di **modellazione**, producendo schemi per **comprendere e spiegare** quello che viene percepito dai sensi.
- La realtà è un'istanza del modello.

- Per **comprendere** il soggetto in analisi.
- Per **conoscere** il soggetto in analisi (fissando ciò che si è compreso).
- Per **comunicare** la conoscenza del soggetto.

I progetti software sono complessi

- Il tipico progetto software **raramente coinvolge un solo sviluppatore**, a volte può coinvolgerne anche centinaia:
 - separare compiti e responsabilità
 - raggruppare le informazioni a diversi livelli di **granularità**
- Il tipico progetto software subisce un **ricambio di personale** nel corso della sua storia:
 - il progetto perde **conoscenza**
 - nuovi sviluppatori devono acquisirla

I progetti software sono complessi

- Le **caratteristiche** del progetto spesso mutano col tempo:
 - necessità di **comunicare** con il cliente in termini chiari
 - **prevedere ed adattarsi** ai cambiamenti
 - **stimarne** l'impatto su costi, tempi e risorse di sviluppo

Come si può ragionare su questo se non si sa su cosa si sta ragionando?

- Un **linguaggio di modellazione** fornisce le primitive a cui ricondurre la realtà in esame.
- Permette di **esprimere** le entità che compongono un sistema complesso, le loro caratteristiche e le relazioni che le collegano.
- Nell'ambito di un progetto, il **linguaggio di modellazione** è normalmente distinto dal processo di sviluppo:
 - il **linguaggio** descrive cosa deve essere ottenuto
 - il **processo** descrive i passi da intraprendere per ottenerlo
 - insieme, **linguaggio e processo** definiscono un metodo di sviluppo

- Agli inizi degli anni '90 vi era una **proliferazione** di linguaggi e approcci alla **modellazione object-oriented**.
- Mancava uno **standard** accettato universalmente per la creazione di modelli software.
- C'era la sensazione che la quantità di differenti soluzioni **ostacolasse** la diffusione dello stesso metodo object-oriented.
- Nel 1994 due esperti di modellazione della Rational Software Corporation, Grady Booch e James Rumbaugh, unificarono i propri metodi, Booch e OMT (Object Management Technique).
- Nel 1995 si unisce anche Ivar Jacobson con il suo OOSE (Object Oriented Software Engineering), dopo l'acquisizione della sua compagnia Objectory da parte di Rational.

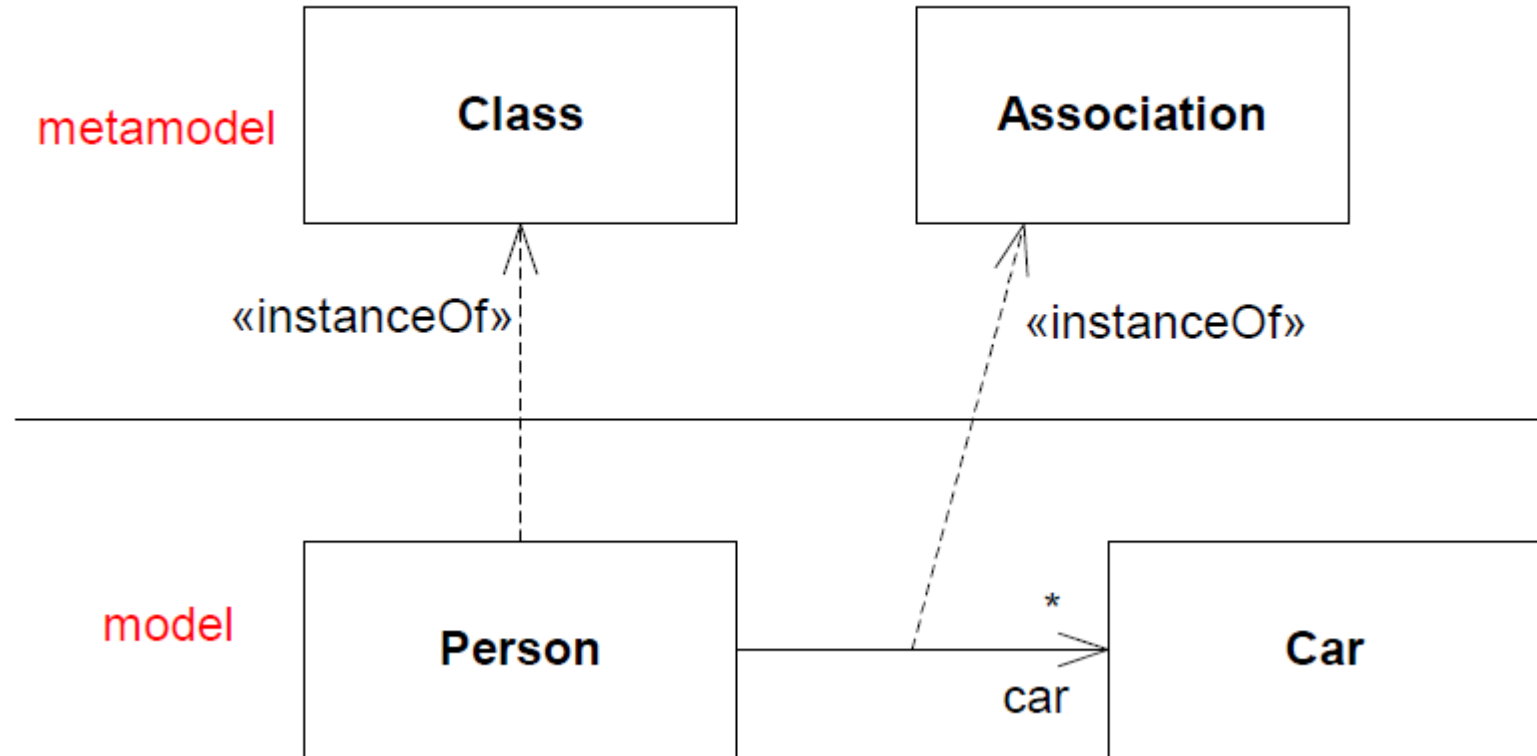
- Nel 1996, Booch, Rumbaugh e Jacobson, noti come i Three Amigos, sono incaricati da Rational di dirigere la creazione dell'**Unified Modeling Language** (UML).
- Nel 1997, la specifica UML 1.0 viene presentata all'OMG (Object Management Group), un consorzio di grandi aziende interessate allo sviluppo di standard e tecnologie basate su oggetti.
- Nel novembre dello stesso anno, una versione arricchita, **UML 1.1**, viene approvata dall'OMG.

- Si può dire che in **UML** tutto è un oggetto.
- La **relazione** classe/istanza costituisce le **fondamenta** stessa del linguaggio.
- UML stesso, il linguaggio, **è un'istanza!**
- UML fa parte di un'**architettura standardizzata** per la modellazione di OMG chiamata **MOF** (Meta-Object Facility).
- Si può vedere **MOF** come un **linguaggio per creare linguaggi**, uno dei quali è **UML**.

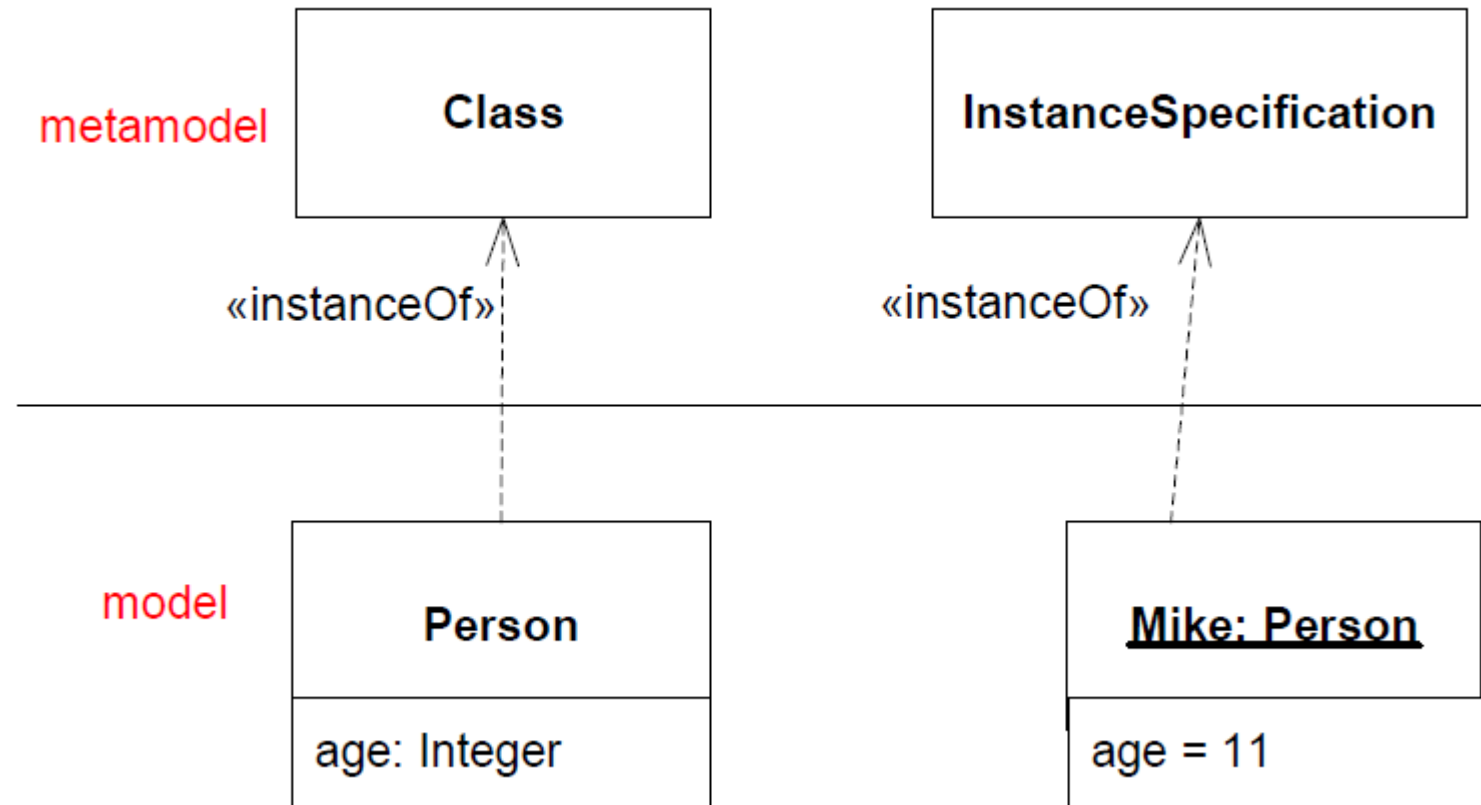
- **MOF ha 4 livelli:** M0, M1, M2 e M3. Ogni livello è un'istanza di un elemento del livello superiore.
 - un elemento di **M0** è la realtà da modellare
 - un elemento di **M1** è un modello che descrive la realtà
 - un elemento di **M2** è un modello che descrive modelli (metamodello); UML è qui
 - un elemento di **M3** è un modello che descrive metamodelli (meta-metamodello); MOF è qui
- OMG usa MOF per definire altri linguaggi oltre a UML.
- Tutti i linguaggi basati su MOF e i modelli con essi prodotti possono essere serializzati e scambiati tramite lo standard XMI (**XML Metadata Interchange**).

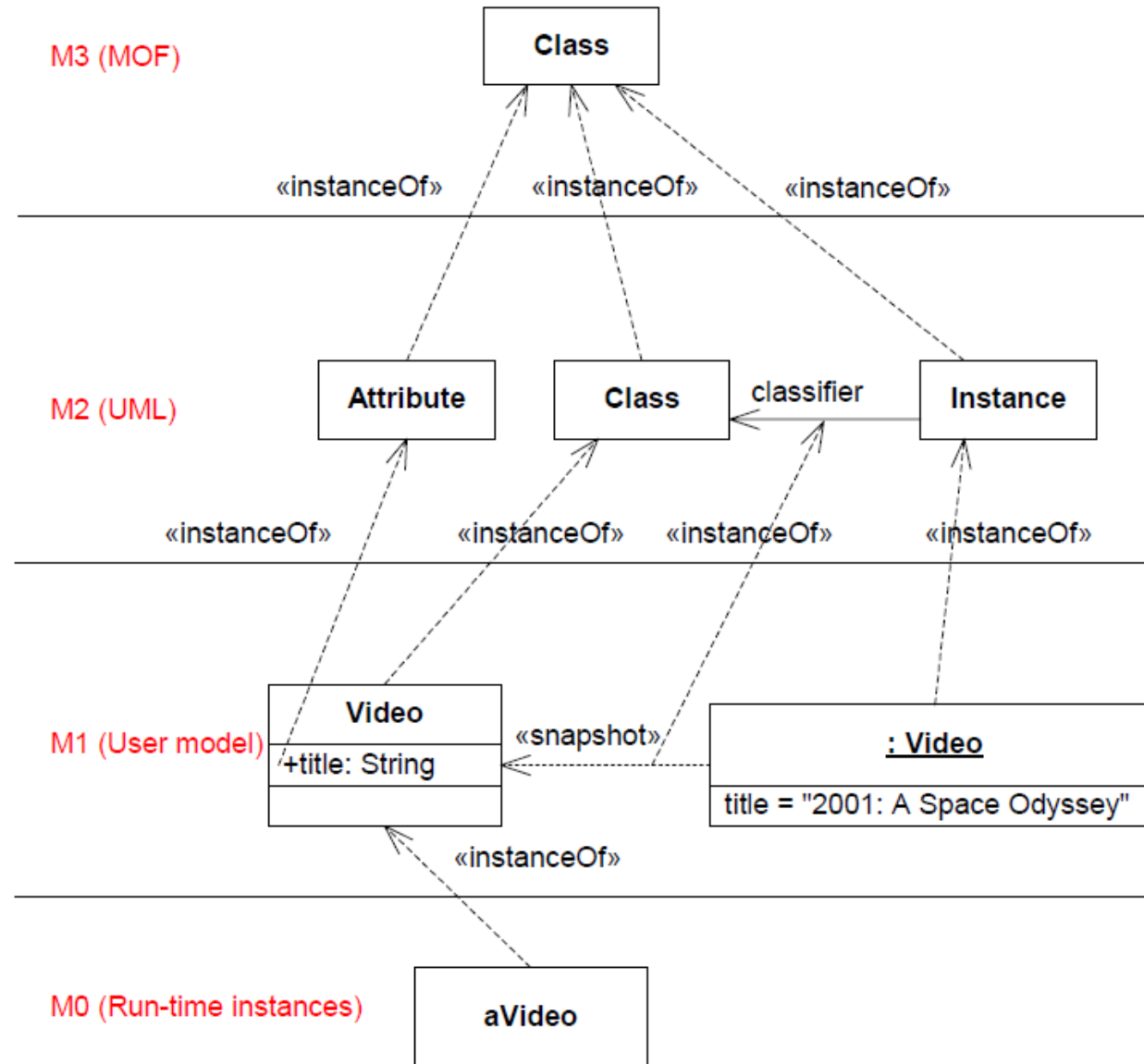
- Si può dire che UML è definito tramite un modello **UML** (o piuttosto, usando un modello M3 che usa primitive comuni a UML).
- In qualunque momento, un oggetto al livello Mx è un'istanza di uno del livello superiore.
- Il **meta-metamodello** di livello M3 è progettato per essere istanza di se stesso, quindi non esiste M4.
- Chi usa **UML** crea modelli di livello M1; tuttavia, è bene sapere che esistono anche i livelli superiori.

M1 e M2 a confronto (usando un diagramma UML).



Altro esempio con M1 (modello utente) e M2 (metamodello UML)





TOPIC 2

Object Oriented Analysis and Design

- UML non è solo un linguaggio per la modellazione, ma un linguaggio per la modellazione orientata agli oggetti.
- Questo include sia l'analisi che la progettazione orientata agli oggetti (OOA e OOD, rispettivamente):
 - **analisi**: capire cosa deve fare il sistema, senza occuparsi dei dettagli implementativi
 - **progettazione**: capire come il sistema raggiunge il suo scopo, come viene implementato
- UML offre strumenti di modellazione OO in entrambi gli ambiti; **frammenti differenti di UML** sono impiegati in diverse fasi del processo di sviluppo (anche se UML stesso non fornisce indicazioni sul suo utilizzo).

- OO: un **paradigma** che sposta l'enfasi della programmazione dal codice verso le entità su cui esso opera, gli oggetti.
- Una rivoluzione **copernicana** cominciata dagli anni '60 e proseguita, lentamente, fino ad affermarsi negli anni '90.
- I **principi cardine** furono proposti separatamente e solo successivamente integrati in unico paradigma.
- Principi **OO** comunemente accettati sono:
 - Abstraction
 - Encapsulation
 - Inheritance
 - Polymorphism.

- **Abstraction** : usare classi per astrarre la natura e le caratteristiche di un oggetto, che è un'istanza della propria classe di appartenenza.
- **Encapsulation** : nascondere al mondo esterno i dettagli del funzionamento di un oggetto; gli oggetti hanno accesso solo ai dati di cui hanno bisogno.
- **Inheritance** : classi possono specializzare altre classi ereditando da esse e implementando solo la porzione di comportamento che differisce.
- **Polymorphism** : invocare comportamento diverso in reazione allo stesso messaggio, a seconda di quale oggetto lo riceve.

- Consente di **modellare** un sistema attraverso l'interazione tra gruppi di oggetti.
- Si tratta di un modo intuitivo di rappresentare la realtà, per mezzo di
 - OGGETTI
 - Scambio di MESSAGGI
 - RESPONSABILITÀ

- Un oggetto è l'elemento base della programmazione ad orientata agli oggetti (OOP).
- Può essere definito come: Contenitore di attributi(le variabili) e comportamenti(i metodi), che agiscono sugli attributi o su altri oggetti.
- Gli attributi possono essere tipi di dato fondamentali, altri oggetti o strutture dati.

- Facilita la realizzazione di modelli “eccellenti” grazie a
 - Incapsulamento (information hiding)
 - Astrazione ed ereditarietà (generalizzazione)
 - Modularizzazione
 - Riutilizzo
- Grazie alla OOAD, il sistema da automatizzare viene modellato ad oggetti, così come viene programmato il software che lo automatizza.

- Prima della realizzazione di un qualsiasi sistema complesso è necessario progettare il software
- I linguaggi di programmazione servono solo all'implementazione.
- E' necessario utilizzare un linguaggio di modellazione che consenta di modellare e progettare software complessi.
- NB: Si progetta prima di implementare.

- Chi progetta un qualsiasi tipo di costruzione o artefatto utilizza sempre figure, schemi, diagrammi per svolgere la propria attività:
 - ingegneri
 - architetti
 - ... ma anche stilisti
- Si utilizzano diagrammi e figure per visualizzare i propri progetti

- Anche i progettisti e gli analisti di sistemi informativi utilizzano figure e diagrammi per visualizzare il risultato del loro lavoro:
 - Modellazione dati
 - Modellazione software
- Progettazione visuale nonostante il prodotto finale risultante non sia necessariamente visuale.

- Il progettista di un sistema software ha la necessità di rappresentare i diversi aspetti del progetto
- Diagrammi differenti, focalizzati sui vari aspetti
- Analogamente al progetto di un edificio
 - Visione completa da più punti di vista: es. Collegamento tra le componenti software
 - Visione di dettaglio e particolari: es. Sequenze di comunicazione

- Tramite il concetto di astrazione una realtà anche molto complessa viene rappresentata semplificandola in un modello.
- Durante analisi e progettazione vengono generati modelli che consentono di identificare ed isolare le caratteristiche di un sistema.
- Il progettista sceglie le caratteristiche rilevanti e definisce le relazioni tra queste.

- Devono essere considerati diversi tipi di relazione
 - Strutturali: es. Associazioni tra classi
 - Temporali: es. Sequenza di messaggi
 - Causa-Effetto: es. Diagramma di stato
 - Organizzative: es. Raggruppamento degli elementi
 - Evolutive: es. Dipendenze tra i diagrammi

TOPIC 3

Unified Modeling Language (UML)

- Unified Modeling Language (UML) è un linguaggio di modellazione visuale
- Consente ad analisti e progettisti di sistemi orientati agli oggetti di modellare, rappresentare e documentare sistemi software.
- Non è un linguaggio di programmazione, tuttavia esistono software per convertire diagrammi UML in elementi di software da espandere
- Non è una metodologia di sviluppo del software.
- UML è un linguaggio, un insieme di elementi e regole, di specifica formale.
- Gli elementi sono forme grafiche che rappresentano ciò che si sta modellando.
- Le regole spiegano come combinare gli elementi.

- UML è un linguaggio semiformale e grafico (basato su diagrammi) per specificare, visualizzare, costruire e documentare gli artefatti di un sistema software.
- Permette di:
 - modellare un dominio
 - scrivere i requisiti di un sistema software
 - descrivere l'architettura del sistema
 - descrivere struttura e comportamento di un sistema
 - documentare un'applicazione
 - generare automaticamente un'implementazione
- Gli stessi modelli UML sono quindi artefatti usati per sviluppare il sistema e comunicare con il cliente (ma anche con progettisti/sviluppatori/etc.)

- Si tratta di un linguaggio di modellazione usato per capire e descrivere le caratteristiche di un nuovo sistema o di uno esistente.
 - Indipendente dall'ambito del progetto.
 - Indipendente dal processo di sviluppo.
 - Indipendente dal linguaggio di programmazione (progettato per essere abbinato alla maggior parte dei linguaggi object-oriented).
- Fa parte di un metodo di sviluppo, non è esso stesso il metodo.

- Si tratta di un vero e proprio linguaggio, non di una semplice notazione grafica.
- Un modello UML è costituito da un insieme di elementi che hanno anche una rappresentazione grafica.
- Il linguaggio è semiformale perché descritto in linguaggio naturale e con l'uso di diagrammi, cercando di ridurre al minimo le ambiguità.
- Ha regole sintattiche (come produrre modelli legali) e regole semantiche (come produrre modelli con un significato).

Tre tipi di regole

- Sintassi astratta
 - Diagrammi e linguaggio naturale
- Regole sintattiche
 - Object Constraint Language (OCL) e linguaggio naturale
- Semantica
 - Linguaggio naturale con supporto di diagrammi

- UML è il linguaggio standard di modellazione più diffuso nello sviluppo di software a livello industriale.
- E' uno standard in evoluzione riconosciuto dall'Organizzazione Internazionale per la Standardizzazione(International Organization for Standardization, ISO).
- Versione attuale: 2.5.1 (Dicembre 2017)
- Sviluppato da Object Management Group (www.omg.org)

- Lo scopo di UML è di definire in modo formale un sistema.
- I progettisti di UML hanno definito un modo per procedere nel processo di sviluppo utilizzando UML.
- Processo Unificato per lo Sviluppo del Software
 - Unified Software Development Process (USDP)
 - Coinvolge persone, progetti, strumenti, processi e prodotti

- Nel processo unificato:
 - i partecipanti e gli sviluppatori coinvolti nel progetto di sviluppo di un sistema, seguono un determinato processo, utilizzando strumenti di ausilio nello sviluppo, generando prodotti software.
- Il processo parte dai requisiti dell'utente, che vengono raccolti nei cosiddetti casi d'uso, delle sequenze di esecuzione del sistema in grado di fornire un valore all'utente.

- Architetto-centrico
 - l'architettura del sistema viene sviluppata in modo da soddisfare i requisiti dei casi d'uso più importanti.
- Iterativo
 - Il progetto viene scomposto in sottoprogetti che costruiscono parti del sistema finale
- Incrementale
 - Il sistema viene costruito incrementalmente unendo le singole parti sviluppate nei sottoprogetti.

- Gli sviluppatori, partendo dai casi d'uso raccolti, producono i modelli del sistema e le implementazioni che li realizzano.
- Fasi del processo
 - Inizio
 - Elaborazione
 - Costruzione
 - Transizione

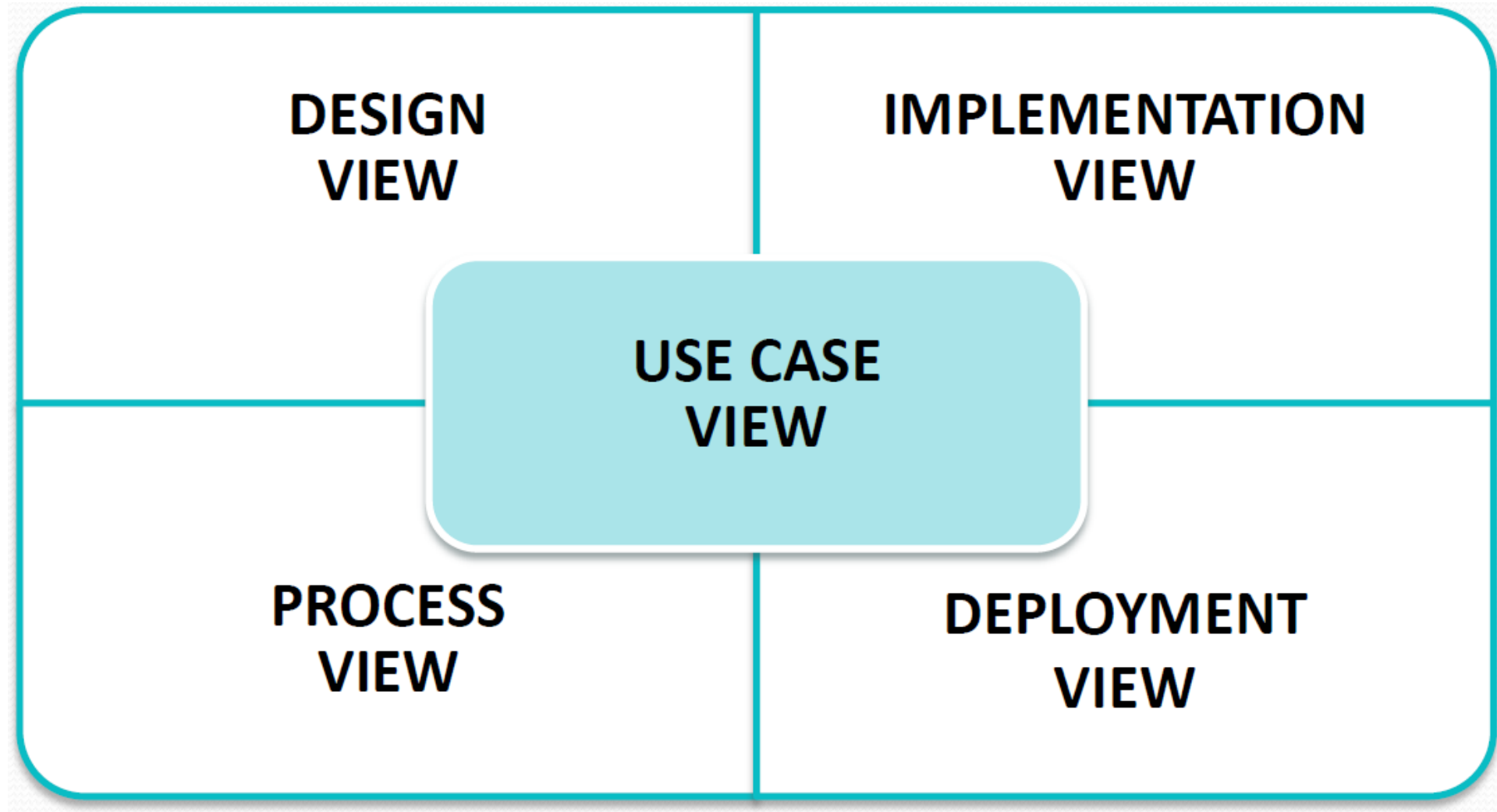
- I prodotti intermedi del processo sono i modelli (astrazioni delle caratteristiche del sistema).
- Ogni modello definisce il sistema da un certo punto di vista, i principali:
 - dei Casi d'Uso
 - di Analisi
 - di Progetto
 - di Deployment
 - di Implementazione
 - di Test

TOPIC 4

Struttura di UML

- Viste
 - Mostrano i differenti aspetti di un sistema attraverso la realizzazione di un certo numero di diagrammi.
 - Sono astrazioni, ognuna delle quali modella il sistema con un'ottica diversa (funzionale, non funzionale, organizzativa).
 - La somma delle viste fornisce il quadro d'insieme.
- Diagrammi
 - Esprimono le viste tramite diagrammi.
- Elementi
 - Sono i concetti che permettono di realizzare i diagrammi (attori).

- In particolare:
 - Un diagramma è la rappresentazione grafica di una parte del modello.
 - Fornisce una vista di un sistema o una sua parte, cioè ne mette in risalto diverse proprietà



- Analizza i requisiti utente
 - Cosa dovrà fare il sistema?
- Vista ad alto livello di importanza fondamentale:
 - Guida lo sviluppo delle rimanenti.
 - Stabilisce le funzionalità che il sistema dovrà realizzare.
- Le funzionalità saranno individuate assieme al cliente grazie all'ausilio di tale vista, utile al reperimento dei requisiti.
- A questo livello è necessario concentrarsi sul cosa fare, astraendosi il più possibile dal come verrà implementato.
- E' la vista che agisce da 'collante' per le altre; spiega il funzionamento desiderato del sistema

- Descrive come realizzare le funzioni
 - Si analizza il sistema dall'interno.
- Si trovano
 - struttura statica del sistema
 - diagramma delle classi
 - diagramma degli oggetti
 - collaborazione dinamica
 - interazioni tra gli oggetti del sistema
- Mette in risalto la scomposizione logica del sistema tramite classi, oggetti e loro relazioni

Implementation view (o Development View)

- Descrive l'aggregazione in moduli, chiamati package, e le relative interdipendenze
- In altre parole mostra l'organizzazione del sistema in blocchi strutturali (package, sottosistemi, librerie, ...)

- Analizza gli aspetti non funzionali del sistema
 - Individuazione dei processi
- Serve a massimizzare l'utilizzo efficiente delle risorse
 - Stabilendo l'esecuzione parallela di determinate operazioni
 - Consentendo la gestione di eventi asincroni
- Mostra i processi (o thread) del sistema in funzione, e le loro interazioni

Deployment view (o Physical View)

- Mostra l'architettura fisica del sistema
- La distribuzione e l'ubicazioni delle componenti software all'interno della struttura stessa.
- Mostra come il sistema viene installato ed eseguito fisicamente

- Usiamo un semplice diagramma dei Casi d'Uso come esempio.
- Regola sintattica: una relazione tra un attore e un caso d'uso può opzionalmente includere una freccia.
- Regola semantica: la freccia significa che la prima interazione si svolge nel senso indicato dalla freccia.



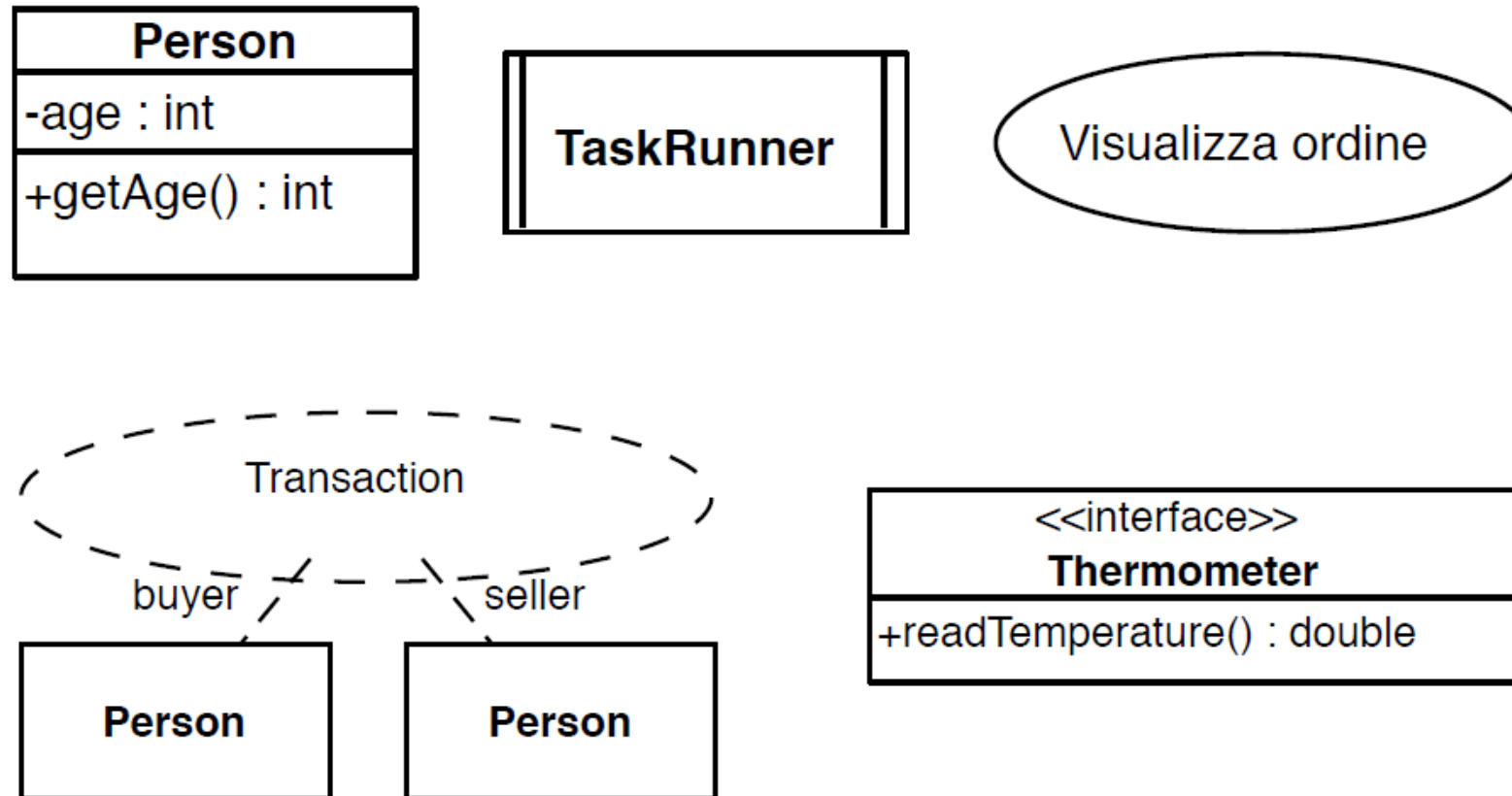
TOPIC 5

Entità UML

- UML prevede diversi tipi di entità che possono essere organizzati in quattro categorie:
 - Strutturali
 - Comportamentali
 - Informative
 - Raggruppamento e contenimento
- Segue una panoramica delle entità principali ma andremo in dettaglio quando analizzeremo i diversi diagrammi

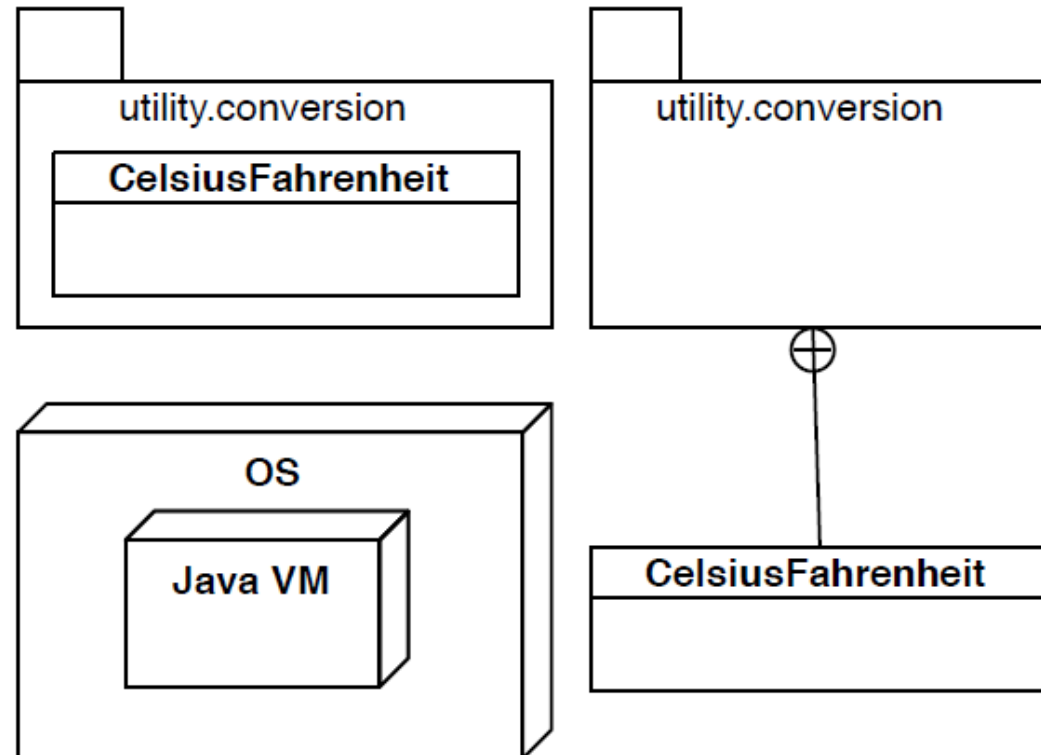
UML: entità strutturali

- Definiscono le «cose» (classificatori, things) del modello Alcuni esempi: classi, classi attive, use-case, collaborazioni, interfacce

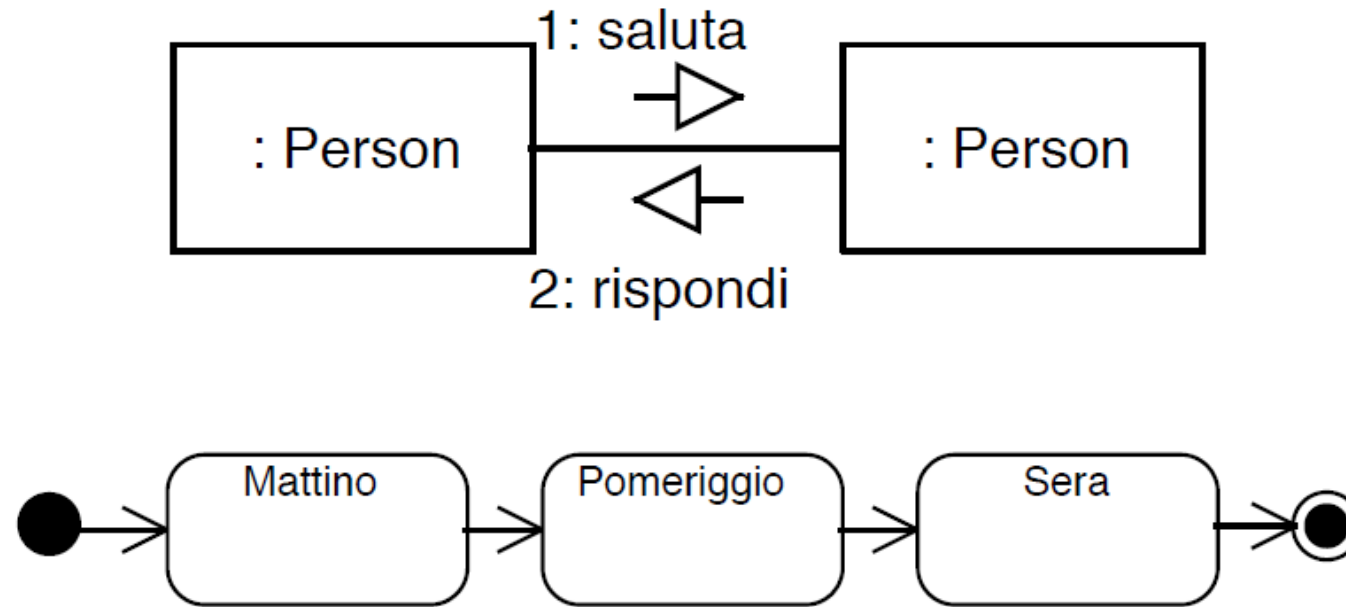


UML: entità di raggruppamento e contenimento

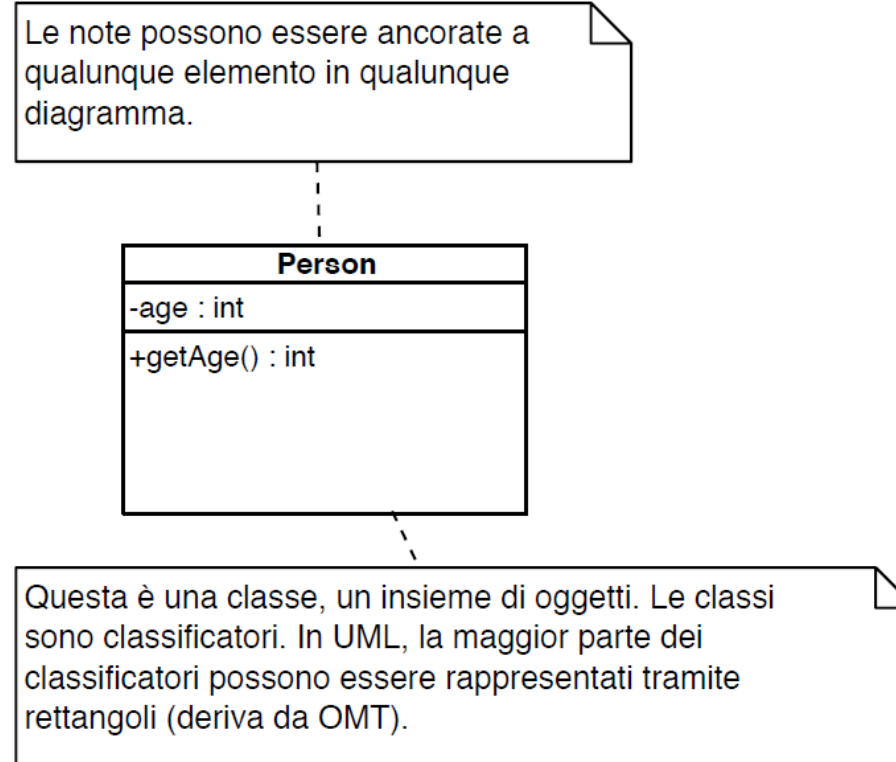
- I package raggruppano altri elementi e forniscono loro un namespace (che permette poi di identificare ogni elemento con il suo nome).
- In UML, moltissimi elementi possono contenere altri elementi al loro interno, formando una struttura gerarchica, rappresentabile graficamente in vari modi.



- Descrivono il "behavior": interazioni, collaborazioni (communication), scambi di messaggi, transizioni di stato, etc.



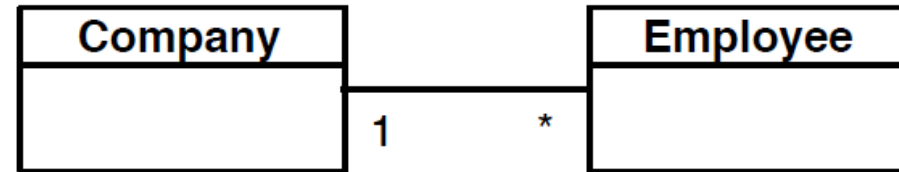
- Uno degli scopi principali della modellazione è la leggibilità: un diagramma non leggibile e informativo serve a poco...
- Le note UML non hanno effetti sul modello ma migliorano la leggibilità.



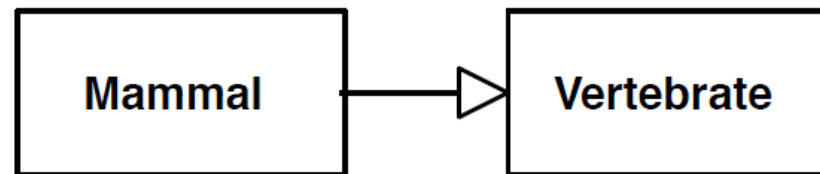
- Gli elementi del modello possono essere collegati da relazioni.
- Rappresentate graficamente tramite linee.
- Possono avere un nome.
- Quattro sottotipi fondamentali:
 - Association
 - Generalization
 - Dependency
 - Realization

Association e generalization

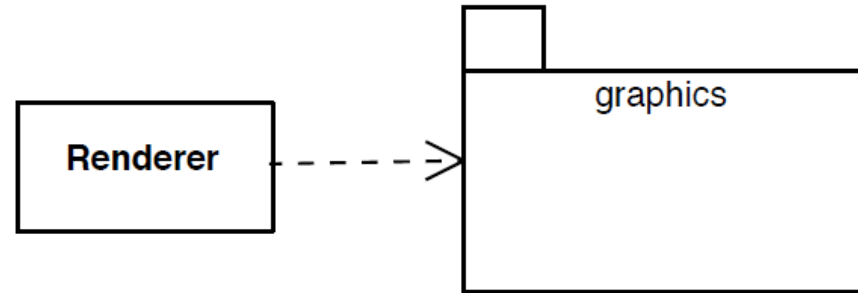
- Un'associazione descrive l'esistenza di un nesso tra le istanze di classificatori (things) e ha varie caratteristiche, alcune opzionali.



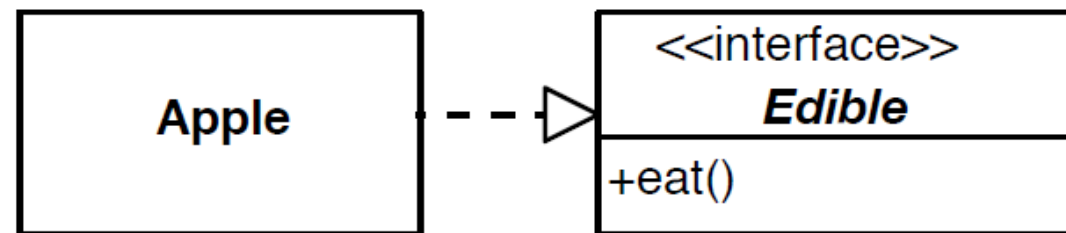
- La generalizzazione è una relazione tassonomica da un elemento specializzato verso un altro, più generale, dello stesso tipo. Il figlio è sostituibile al genitore dovunque appaia, e ne condivide struttura e comportamento.



- Una dipendenza è una relazione semantica: indica che il client dipende, semanticamente o strutturalmente, dal supplier (variazioni alla specifica del supplier possono cambiare quella del client).



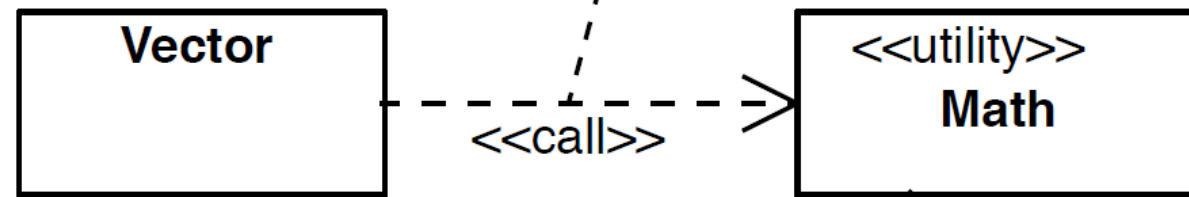
- Anche la realizzazione è una relazione semantica: il supplier fornisce una specifica, il client la realizza (es: implementazione di interfacce, templates, etc.)



- Un metodo mnemonico per ricordarsi il verso di tutte le frecce in UML è il seguente:
 - In UML, tutte le frecce vanno da chi sa verso chi non sa (dell'esistenza dell'altro).
 - In una generalizzazione, il figlio sa di estendere il genitore, ma il genitore non sa di essere esteso.
 - In una dipendenza, chi dipende sa da chi dipende, ma non vice-versa.
 - In una realizzazione, chi implementa conosce la specifica, ma non il contrario.

- Un'altra primitiva di UML comune ad ogni diagramma.
- Rende un diagramma più informativo arricchendo la semantica dei costrutti UML.
- Uno stereotipo è una parola chiave tra virgolette e abbinata ad un elemento del modello.
- Esempi:
 - «import»
 - «utility»
 - «interface»

Questa dipendenza tra Vector e Math ha lo stereotipo «call»; significa che operazioni di Vector invocano operazioni di Math.



Lo stereotipo «utility» indica che questa è una classe utility, cioè una collezione di variabili globali e operazioni statiche usate da altre parti del sistema.

- Gli stereotipi forniscono significato aggiuntivo ai costrutti UML.
- Possono essere usati per adattare UML a particolari ambiti e piattaforme di sviluppo.
- Stereotipi, vincoli e regole aggiuntivi vengono raccolti in profili, che costituiscono uno dei principali meccanismi di estensione di UML.

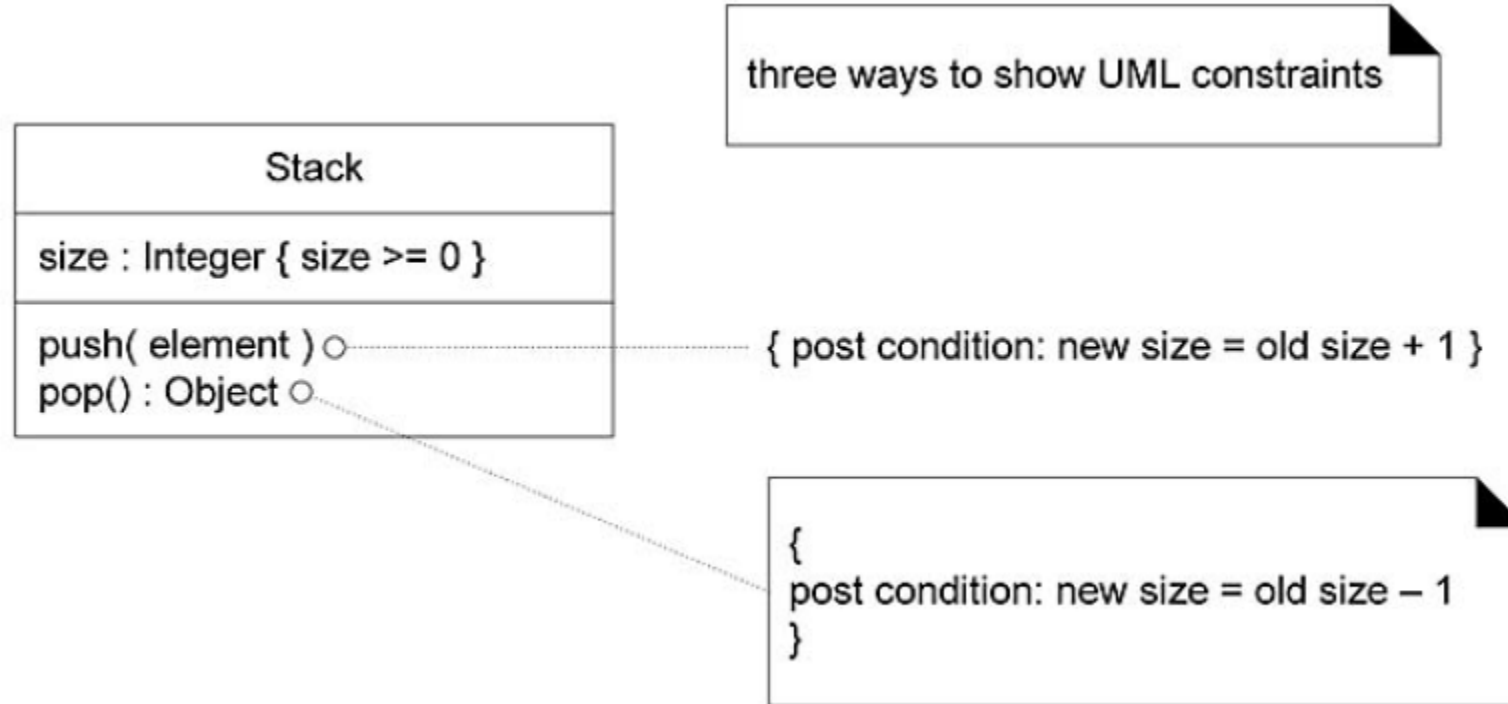
- Object Constraint Language (OCL) è un linguaggio, approvato e standardizzato da OMG, per la specifica di vincoli.
- Si usa assieme ad UML e a tutti i linguaggi dell'architettura MOF.
- Non è obbligatorio, ma aggiunge rigore formale al modello.
- Specifica condizioni che devono essere soddisfatte dalle istanze di una classe: i vincoli sono espressioni booleane considerate true.
- Un tool UML può usare OCL
 - per validare il modello
 - per generare codice

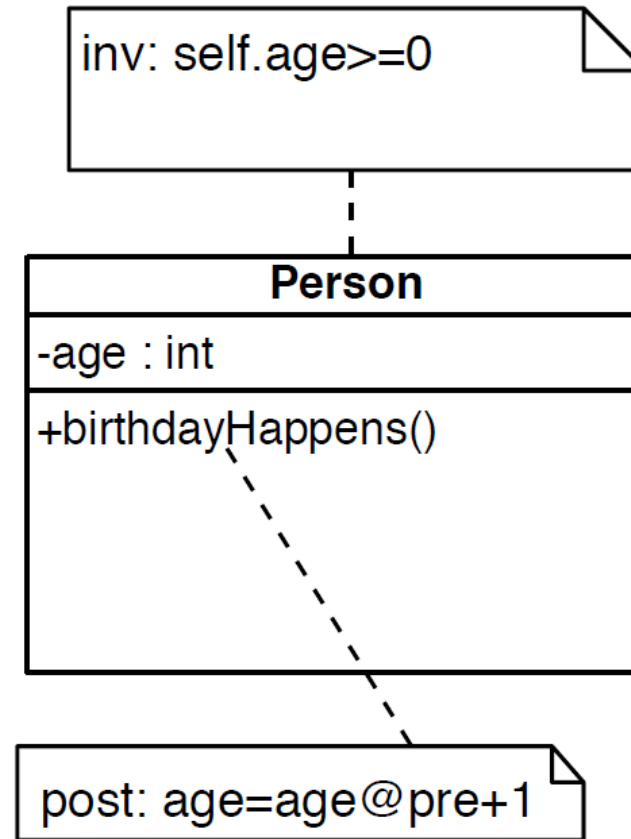
- Un vincolo OCL opera in un determinato contesto, specificando proprietà soddisfatte da tutte le istanze di quel contesto.
- Il contesto può essere una classe, un suo attributo o una sua operazione.
- I vincoli hanno un tipo che descrive l'ambito della loro validità.
- Esempio di vincolo:

context Car inv: fuel>=0

- Questo vincolo si applica alla classe Car ed è un invariante (sempre valido).

- inv: Invariante, sempre valido nel contesto.
- pre: Nel contesto di un'operazione, una preconditione per la sua esecuzione.
- post: Nel contesto di un'operazione, una postcondizione vera dopo l'esecuzione.
- body: Definisce una query nel contesto.
- init: Definisce il valore iniziale nel contesto.
- derive: Definisce un attributo derivato del contesto.





In una postcondizione, '@pre' permette di accedere al valore precedente di un attributo.)

- Modellare è indispensabile in qualunque progetto non banale.
- UML è un linguaggio general-purpose per la modellazione.
- Non è perfetto, ma è potente e costituisce uno standard diffuso ed accettato.
- Costruire un buon modello è difficile.

- Visual Paradigm: <https://www.visual-paradigm.com/>
- ArgoUML: <http://argouml.tigris.org/>
- Papyrus: <http://eclipse.org/papyrus>
- Umbrello: <http://uml.sourceforge.net/>
- Eclipse: www.eclipse.org/uml2/
- ...

TOPIC 6

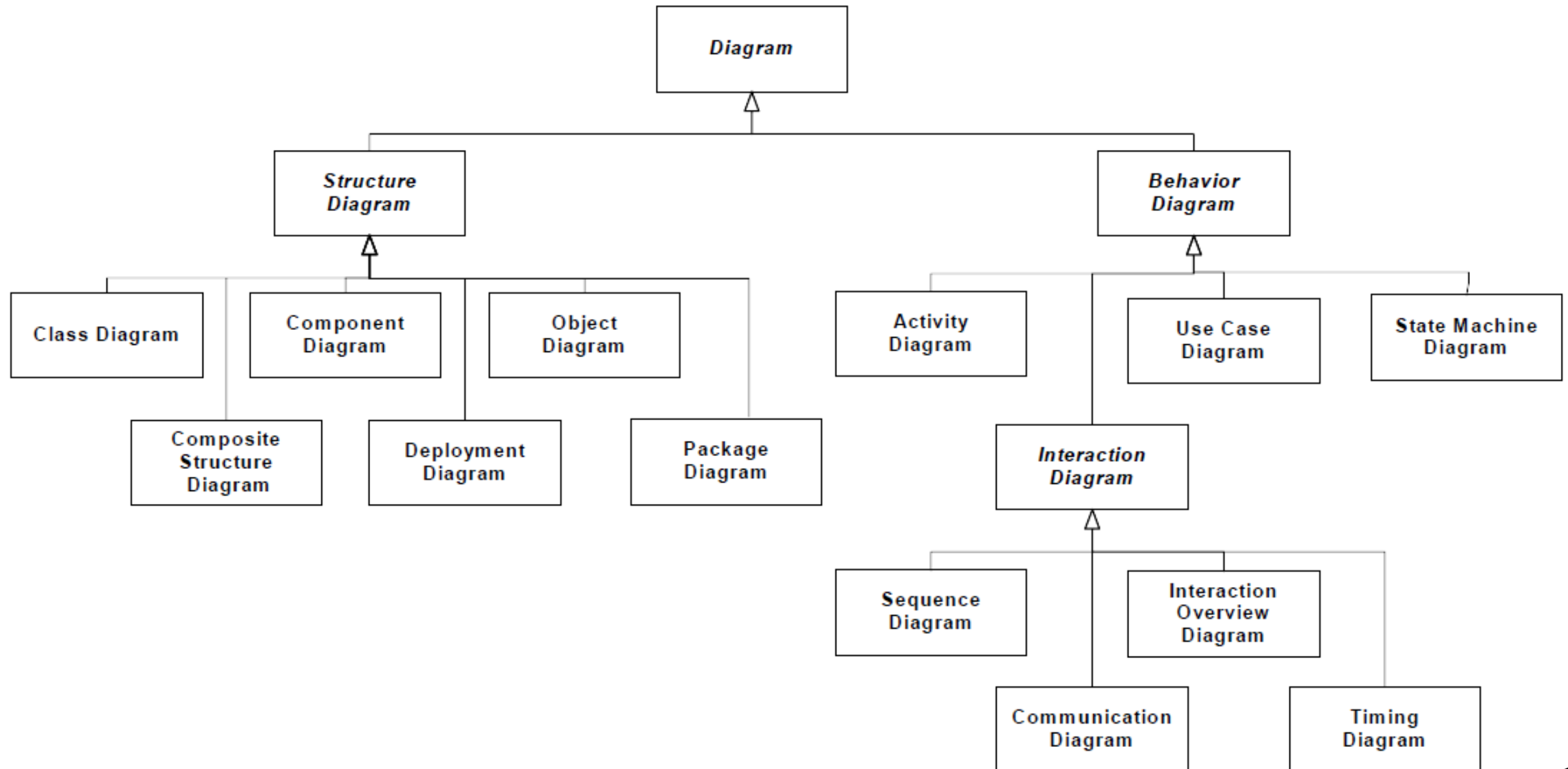
Diagrammi UML

- Sono grafici che visualizzano una particolare proiezione del sistema da modellare. UML 2 definisce 13 diagrammi (contro i 9 di UML 1.x), divisi in due categorie:
 - Structure diagrams
 - Descrivono come è fatto il sistema;
 - Forniscono le viste
 - Logical
 - Development
 - Physical
 - Behavior diagrams:
 - Descrivono come funziona il sistema;
 - Forniscono le viste
 - Process
 - Use case

Structure	Behavior
Class diagram Object diagram Package diagram* Composite Structure diagram* Component diagram Deployment diagram	Use Case diagram Activity diagram State Machine diagram Sequence diagram Communication diagram Interaction Overview diagram* Timing diagram*

* : non esiste in UML 1.x

Tassonomia dei 13 diagrammi di UML 2



TOPIC 7

Casi d'Uso

- I casi d'uso esistono da prima dei diagrammi UML!
- Proposti da Ivar Jacobson nel 1992 per il metodo Objectory
- In realtà la tecnica era già consolidata: «studio degli scenari di operatività degli utilizzatori di un sistema»
- In altre parole:
 - i modi in cui il sistema può essere utilizzato
 - le funzionalità che il sistema mette a disposizione dei suoi utilizzatori

- I requisiti funzionali specificano cosa deve essere fatto.
- Sono indipendenti dalla tecnologia, dall'architettura, dalla piattaforma, dal linguaggio di programmazione.
- I requisiti non-funzionali specificano vincoli aggiuntivi, ad esempio:
 - performance
 - scalabilità
 - tolleranza ai guasti
 - dimensione degli eseguibili
- I casi d'uso modellano i requisiti funzionali.

- Specifica cosa ci si aspetta da un sistema
- Nasconde come il sistema lo implementa
- E' una sequenza di azioni (con varianti) che producono un risultato osservabile da un attore
- Si usa per
 - Descrivere i requisiti (analisi iniziale)
 - Convalidare l'architettura e verificare il sistema

- I desiderata sono ciò che il cliente desidera.
- Formalizzare i desiderata in requisiti è una delle più importanti sfide aperte dell'ingegneria del software.
- La specifica errata o incompleta delle richieste del cliente è una delle cause principali del fallimento dei progetti software.
- Il diagramma e la modellazione dei casi d'uso aiutano l'interazione con il cliente e migliorano l'estrazione dei requisiti (funzionali).

- Cliente:
 - vorrei vendere i manufatti che realizzo...
 - non vorrei solo un mercato locale...
 - mi piacerebbe che gli acquirenti potessero visionare un catalogo da cui scegliere...
 - vorrei gestire gli ordini da qualunque posto perché viaggio molto...
- Che cosa viene fatto? Da chi?

- Cliente:
 - vorrei avere la possibilità di creare un catalogo dei miei manufatti...
 - vorrei un catalogo liberamente consultabile da chiunque...
 - vorrei organizzare i manufatti raccogliendoli in categorie...
 - vorrei che gli interessati all'acquisto potessero inviarmi un ordine, che io provvederò ad evadere previa una qualche forma di registrazione...

- Chi interagisce con il sistema (attori)?
 - Clienti
 - Amministratori del negozio online
 - Reparto ordini
- Cosa fanno (casi d'uso)?
 - Il cliente si registra, consulta il catalogo ed effettua acquisti
 - L'amministratore organizza il catalogo, che è diviso in categorie
 - Il reparto ordini riceve ordini da evadere
 - Il cliente sceglie il tipo di pagamento

- Visione d'insieme del sistema che si sta analizzando
 - Fondamentali nelle fasi iniziali del progetto
- Rappresentano il sistema visto dall'utilizzatore
 - Modellano il dialogo tra l'utilizzatore e il sistema stesso
 - Descrivono l'interazione tra attori e sistema
 - Non la logica propria della funzione (sono astratti)
 - Espressi in forma testuale (comprensibili anche ai non addetti)
 - Diversi livelli di definizione (sistema complessivo o sue parti)
 - Rappresentano le modalità d'uso del sistema da parte di uno o più attori

- Consentono di scoprire i requisiti funzionali
 - Rappresentano un valido ausilio nel dialogo con l'utente
 - Fondamentali con esponenti non tecnici
- Vengono rappresentati da un diagramma.

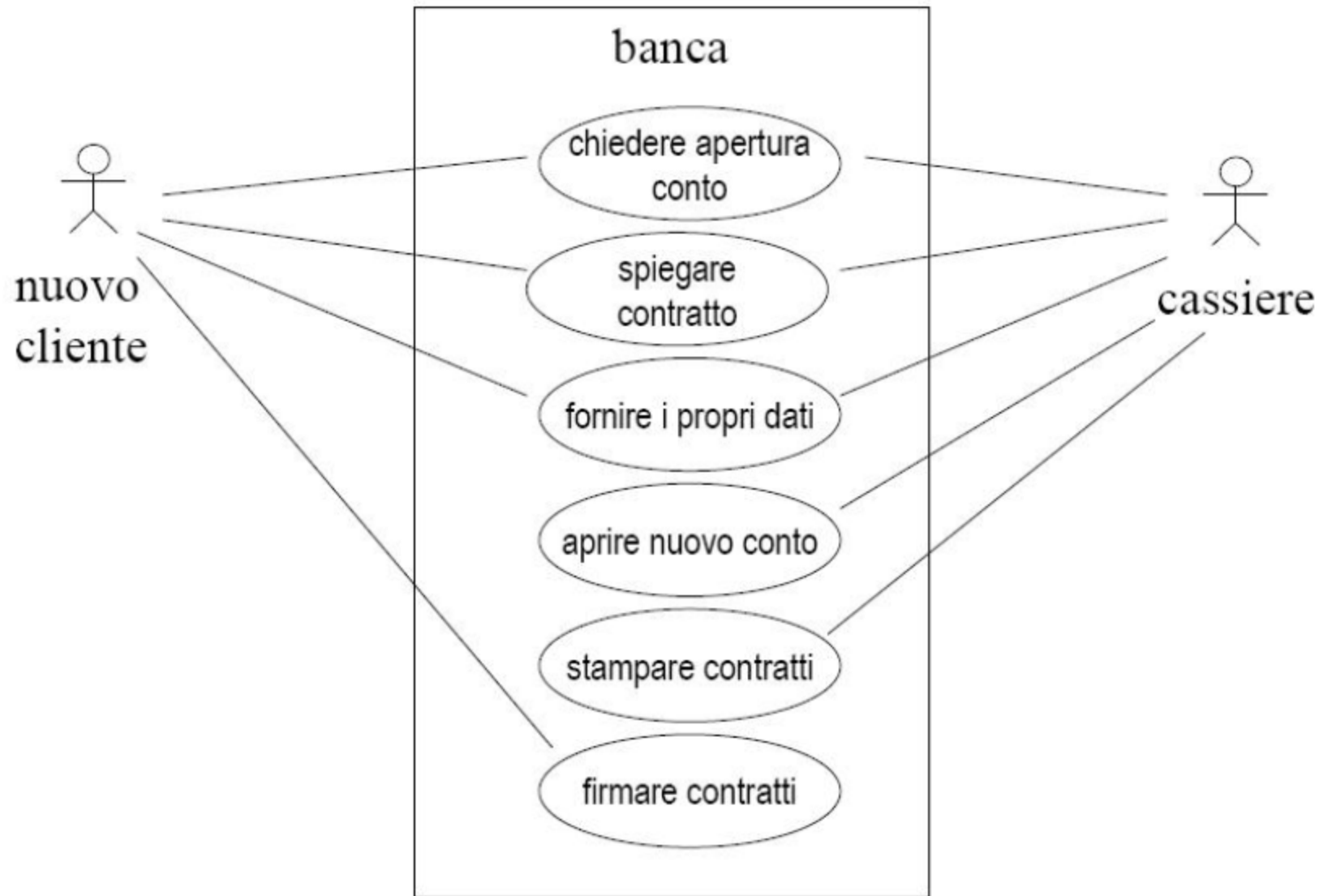
TOPIC 8

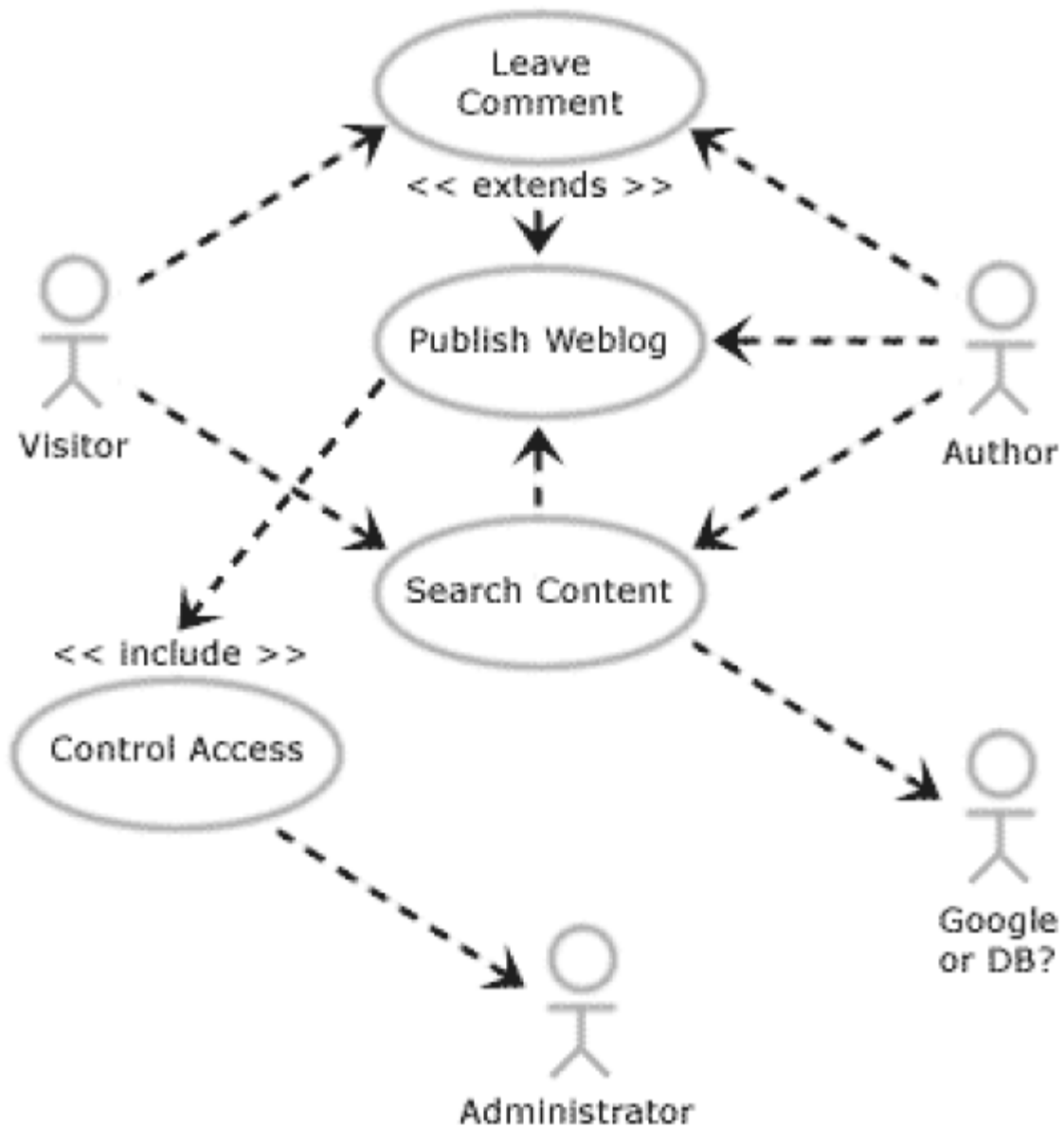
Casi d'Uso

- Rappresenta i casi d'uso, gli attori e le associazioni che li legano.
- Visualizzano le modalità d'utilizzo del sistema da parte di uno o più utilizzatori, che vengono definiti attori.
- I singoli casi d'uso descrivono le iterazioni tra sistema e attori (non la logica delle funzioni).
- Un attore è un elemento esterno al sistema che fornisce un'input a cui il sistema deve rispondere.

- Si tratta di un diagramma che esprime un comportamento, desiderato o offerto.
- L'oggetto esaminato è solitamente un sistema o una sua parte Individua:
 - chi o che cosa ha a che fare con il sistema (attore)
 - che cosa l'attore può fare (caso d'uso).
- Tipicamente è il primo tipo di diagramma ad essere creato in un processo o ciclo di sviluppo, nell'ambito dell'analisi dei requisiti.

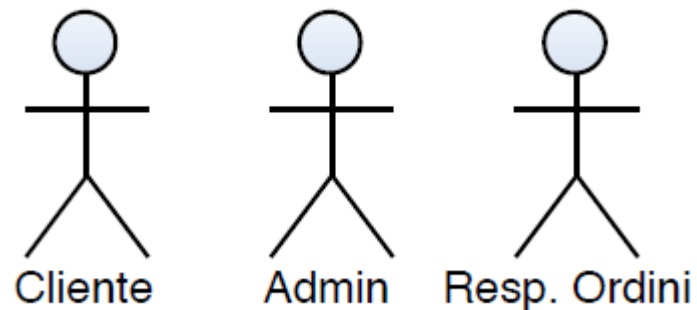
Esempio semplice: conto in banca





- Si tratta di un diagramma che esprime un comportamento, desiderato o offerto.
- L'oggetto esaminato è solitamente un sistema o una sua parte Individua:
 - chi o che cosa ha a che fare con il sistema (attore)
 - che cosa l'attore può fare (caso d'uso).
- Tipicamente è il primo tipo di diagramma ad essere creato in un processo o ciclo di sviluppo, nell'ambito dell'analisi dei requisiti.

- Utilizzatore del sistema che interagisce con i casi d'uso
 - Può essere umano oppure un altro sistema
- Rappresenta un ruolo di un oggetto
 - Un oggetto fisico (classe) può assumere ruoli diversi e quindi essere modellato da più attori.
 - Nel caso di attori umani, il nome di un attore identifica il ruolo che l'attore svolge in quel contesto, non il suo incarico o mansione.



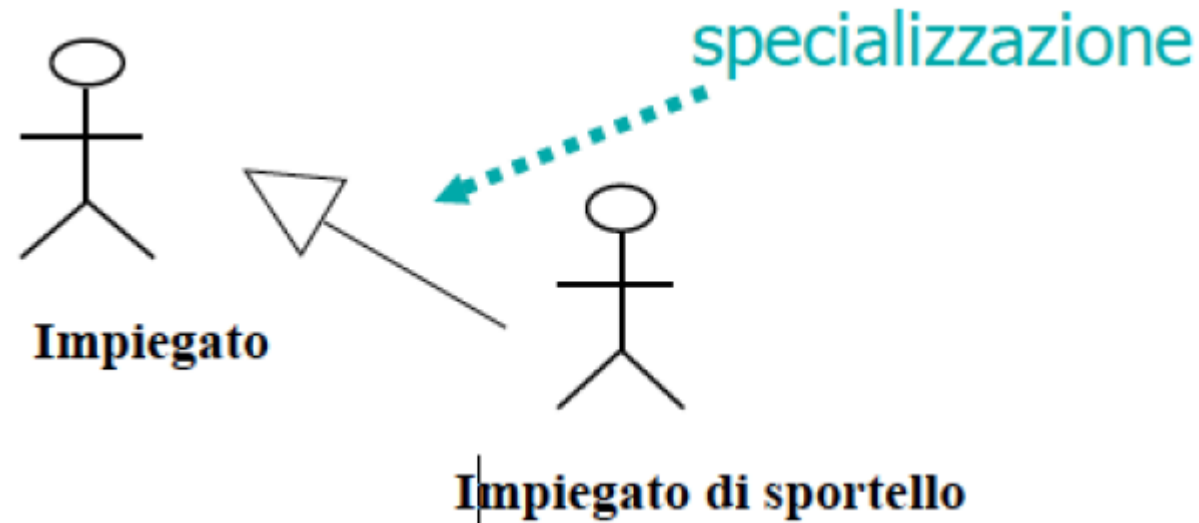
- Un attore specifica un ruolo assunto da un utente o altra entità che interagisce col sistema nell'ambito di un'unità di funzionamento (caso d'uso).
- Un attore è esterno al sistema.
- Non è necessariamente umano:
 - oggetto fisico
 - agente software
 - condizioni ambientali
 - etc.
- Gli attori eseguono casi d'uso
 - prima si cercano gli attori ...
 - ...poi i loro casi d'uso!

Come individuare gli attori

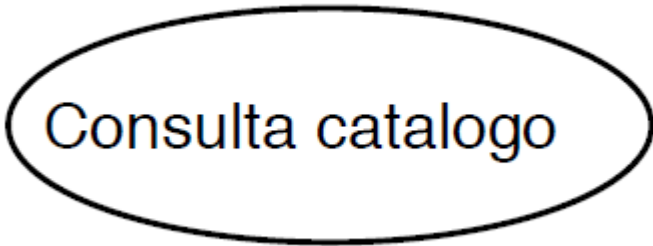
- Per individuare un attore è necessario individuare chi/cosa interagisce col sistema e con quale ruolo.
- Domande utili:
 - Chi/cosa usa il sistema?
 - Che ruolo ha chi/cosa interagisce col sistema?
 - In quale parte dell'organizzazione è utilizzato il sistema?
 - Chi/cosa avvia il sistema?
 - Chi supporterà e manterrà il sistema?
 - Altri sistemi interagiscono col sistema?
 - Ci sono funzioni attivate periodicamente? es. backup
 - Chi/cosa ottiene o fornisce informazioni dal sistema?
 - Un attore ha diversi ruoli? Lo stesso ruolo è assegnato a più attori?

Specializzazione degli attori

- Gli attori possono essere specializzati e connessi ad altri attori tramite relazioni di generalization
- Attori specializzati usano le stesse funzionalità degli attori che generalizzano ed eventualmente alcune funzionalità specifiche.



- Rappresenta una particolare modalità d'utilizzo del sistema.
- Rappresenta alcune delle funzioni visibili all'attore.
- Consente all'attore di raggiungere alcuni obiettivi
 - Manipolazione di informazione
 - Esecuzione di funzione
 - es.: archivia documento



Consulta catalogo

- La specifica di una sequenza di azioni, incluse eventuali sequenze alternative e/o di errore che un sistema (o sottosistema) può eseguire interagendo con attori esterni.
- È qualcosa che un attore vuole il sistema faccia:
 - È descritto dal punto di vista dell'attore
 - È causato da un'azione di un attore
- Il nome (etichetta) dovrebbe essere basato su un verbo o su un sostantivo che esprime un avvenimento

- Un caso d'uso è sempre iniziato da un attore: in UML, un evento è sempre legato all'entità che lo ha generato.
- L'attore che inizia un caso d'uso è detto primario, gli altri attori che interagiscono nell'ambito di quel caso d'uso sono secondari.
- Un caso d'uso è un classificatore dotato di comportamento: può essere specificato da diagrammi di stato, interazione, sequenza, avere pre- e post-condizioni, ...
- Può ammettere al suo interno varianti al comportamento principale.

- Individuare i casi d'uso è un'operazione iterativa.
- Per individuare i casi d'uso possono essere utili le seguenti domande:
 - Ciascun attore che funzioni si aspetta?
 - Il sistema gestisce (archivia/fornisce) informazioni? Se sì quali sono gli attori che provocano questo comportamento?
 - Alcuni attori vengono informati quando il sistema cambia stato?
 - Gli attori devono informare il sistema di cambiamenti improvvisi?
 - Alcuni eventi esterni producono effetti sul sistema?
 - Quali casi d'uso mantengono il sistema?
 - I requisiti funzionali sono tutti coperti dai casi d'uso?

Come recuperare informazioni su un caso d'uso

- Interviste con gli esperti del dominio (desiderata ben strutturati)
- Bibliografia del dominio del sistema Sistemi già esistenti
- Conoscenza personale del dominio

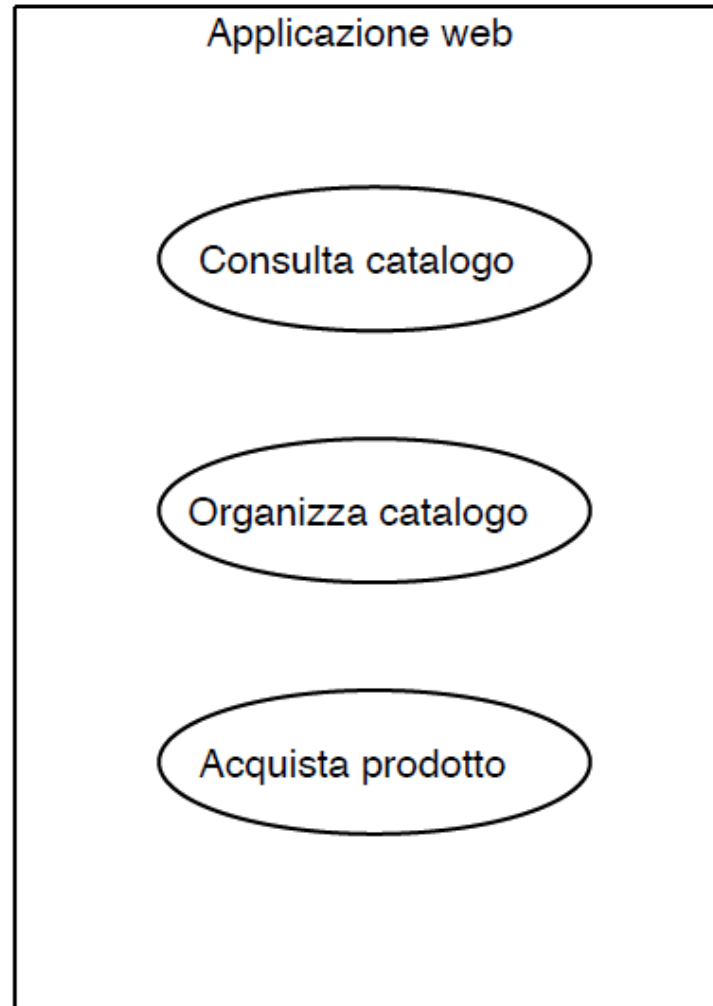
Come descrivere un caso d'uso

- Descrivere in modo generico e sequenziale il flusso di eventi di un caso d'uso
 - Descrivere la preconditione (stato iniziale del sistema)
 - Elencare la sequenza di passi
- Descrivere le interazioni con gli attori e i messaggi scambiati
- Aggiungere eventuali punti di estensione
- Descrivere in modo chiaro, preciso e breve

TOPIC 9

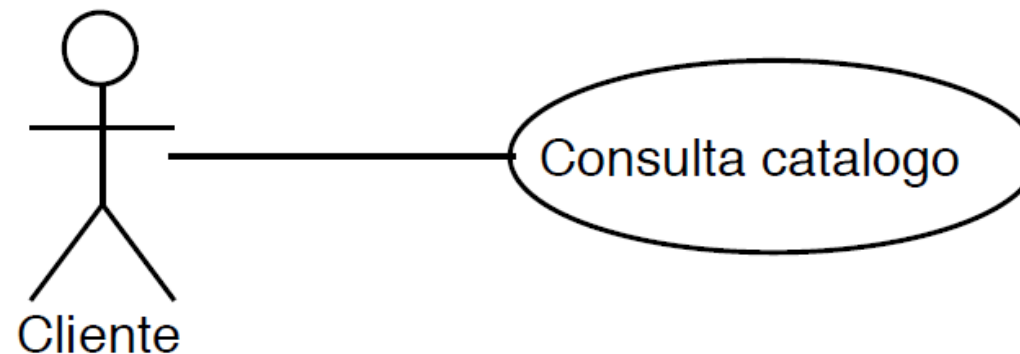
Casi d'Uso: Elementi del diagramma

- Delimita l'argomento del diagramma, specificando i confini del sistema.

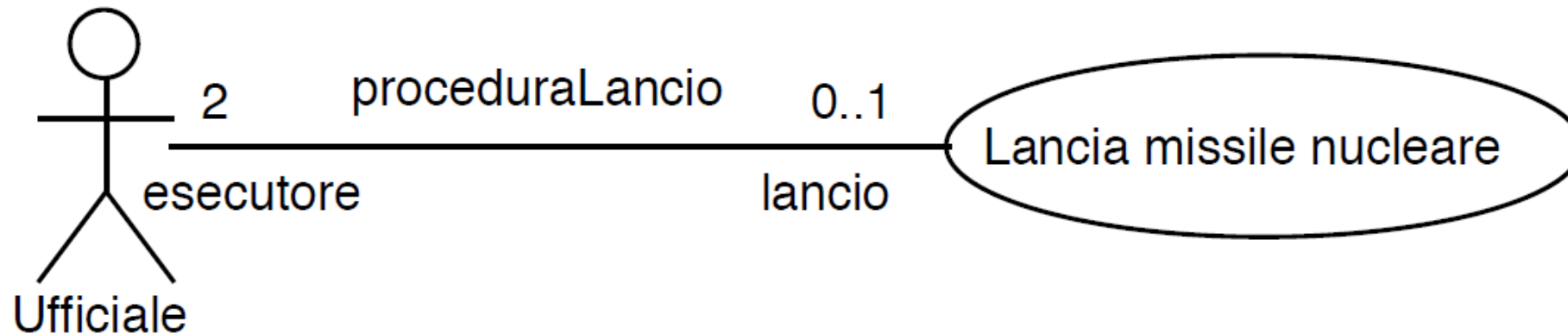


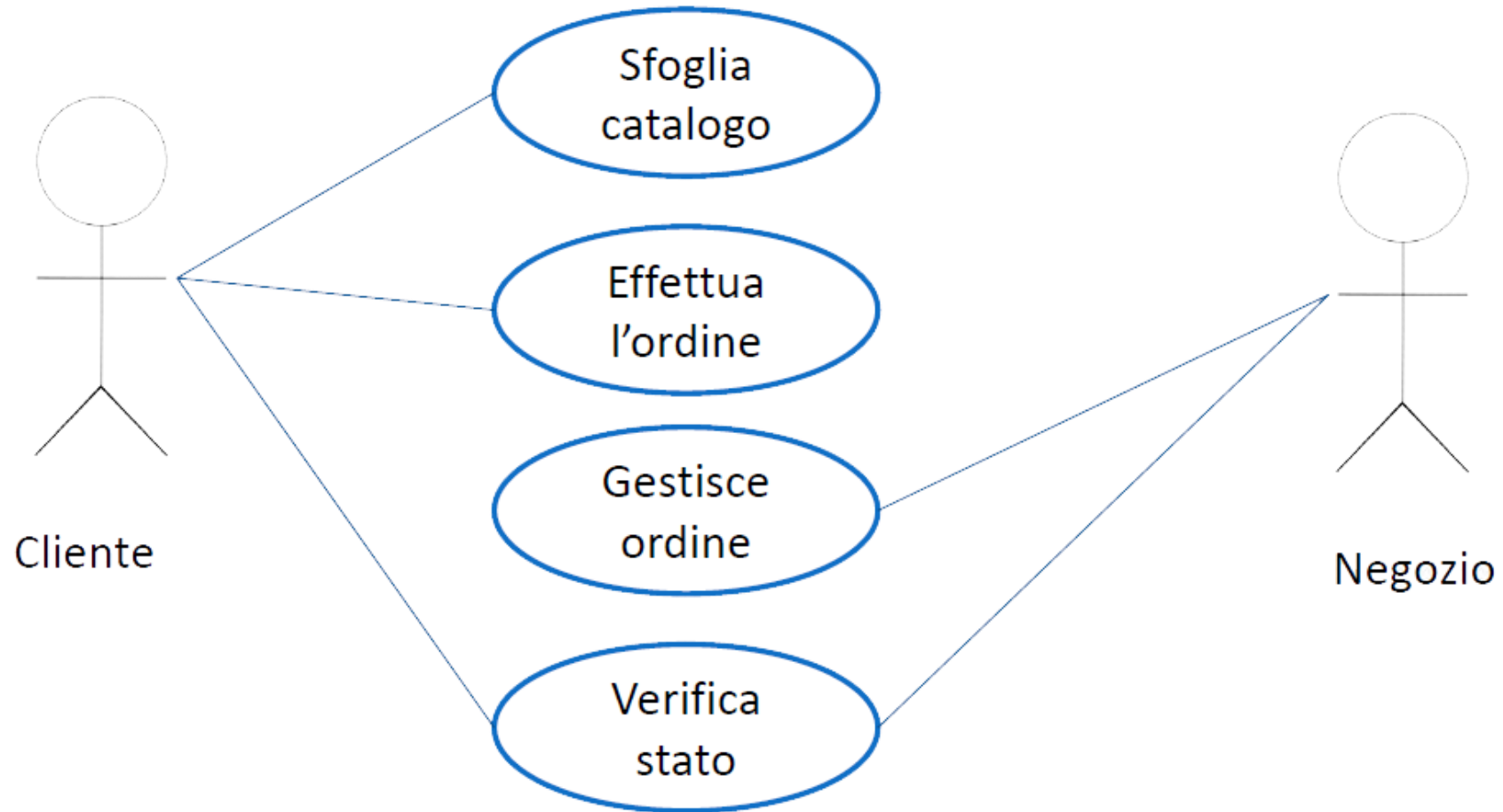
Elementi del diagramma: associazione

- Relazioni (ovvero collegamenti) tra attori e use case; La relazione indica l'esistenza di un'associazione tra un attore ed un caso d'uso:
 - Quindi una particolare persona (o sistema) che si trova in un certo ruolo comunica con specifiche istanze del caso d'uso, partecipando agli eventi rappresentati dal caso.
- Un attore si può associare solo a casi d'uso, classi e componenti (che verranno eventualmente associati ad altri casi d'uso, classi e componenti).
- Un caso d'uso non dovrebbe essere associato ad altri casi d'uso riguardanti lo stesso argomento.



- Alcune caratteristiche opzionali comuni a tutte le associazioni in UML:
 - nome
 - molteplicità
 - ruoli

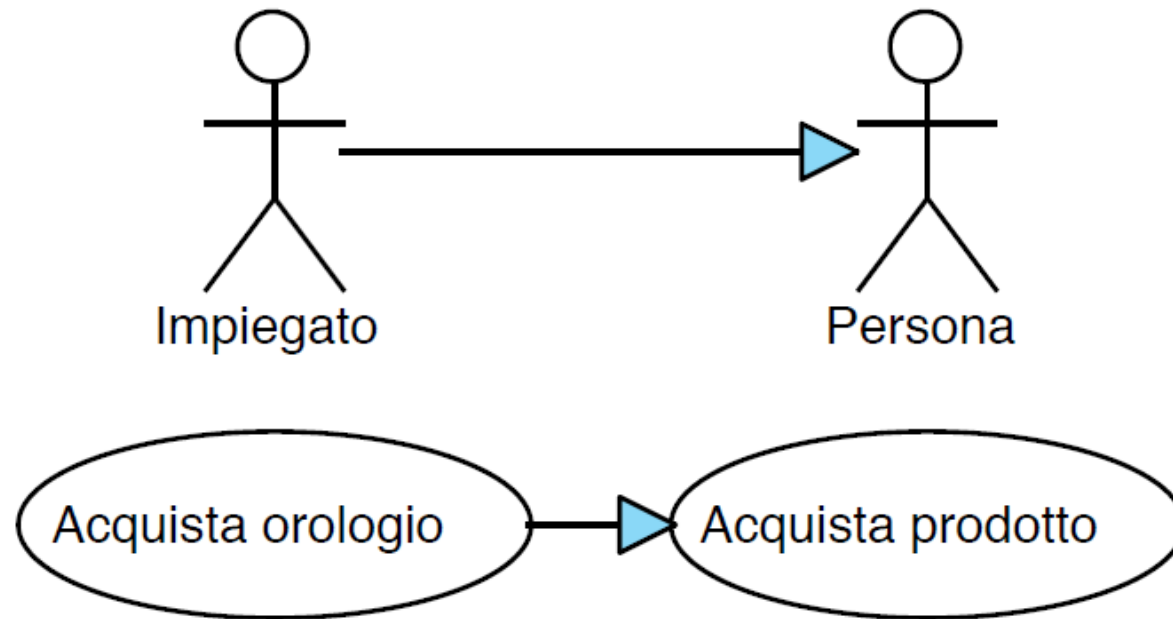




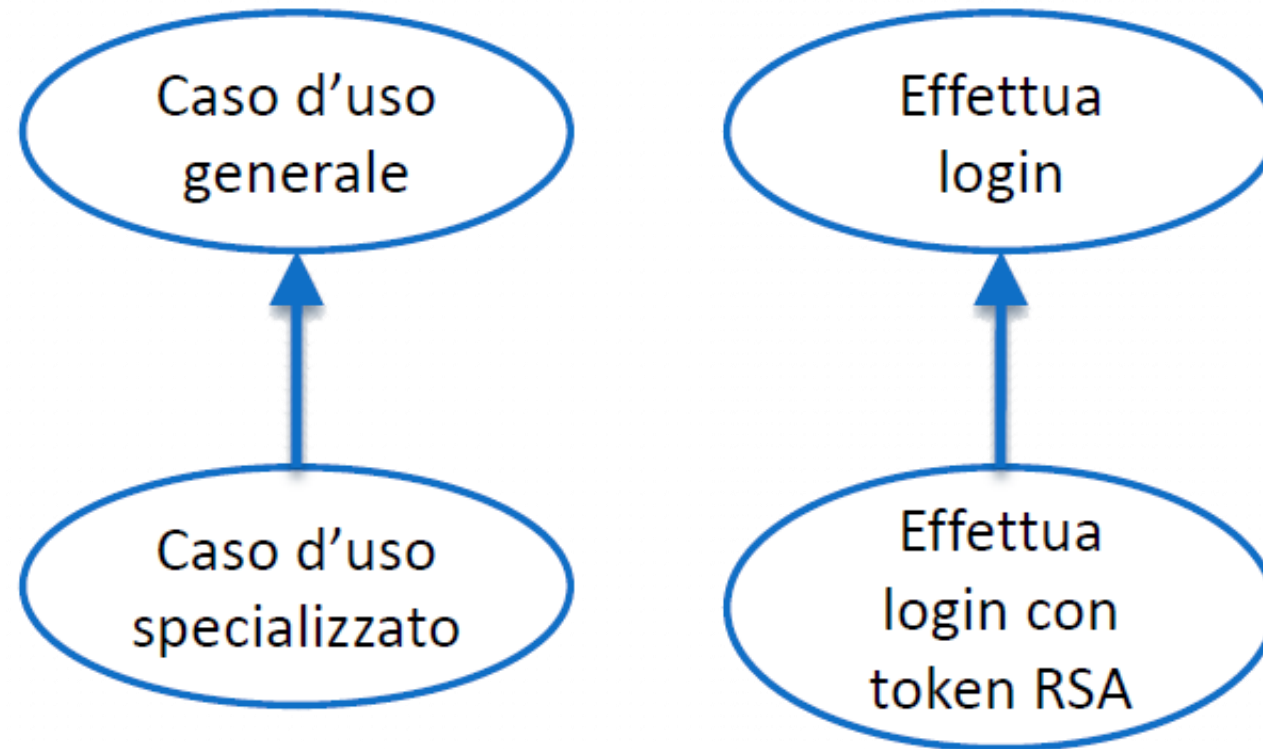
- Con i diagrammi dei casi d'uso si possono inoltre rappresentare:
 - Generalizzazioni tra casi d'uso
 - Generalizzazioni tra attori
 - Relazioni di inclusione tra casi d'uso (include)
 - Relazioni di estensione tra casi d'uso (extend)

Elementi del diagramma: generalizzazione

- Collega un attore o caso d'uso ad un altro più generale.
- Il figlio può sostituire il genitore dovunque questi appaia.

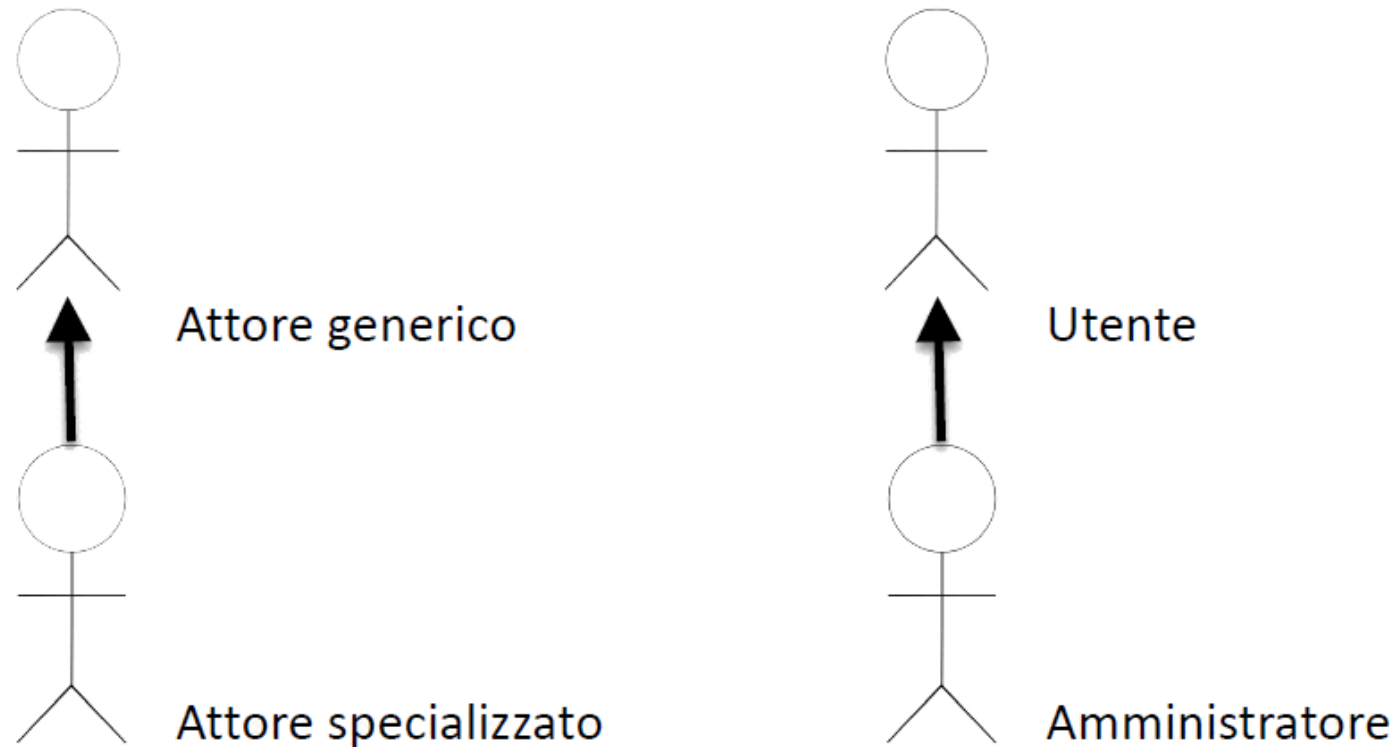


- Può esistere più di una versione di un caso d'uso, aventi ognuna alcune azioni in comune ed altre uniche per ciascun caso.
- Nel diagramma si associano i casi d'uso con una freccia puntata verso il caso d'uso più generale.
- Il caso d'uso specializzato eredita parte delle funzionalità del caso d'uso generico



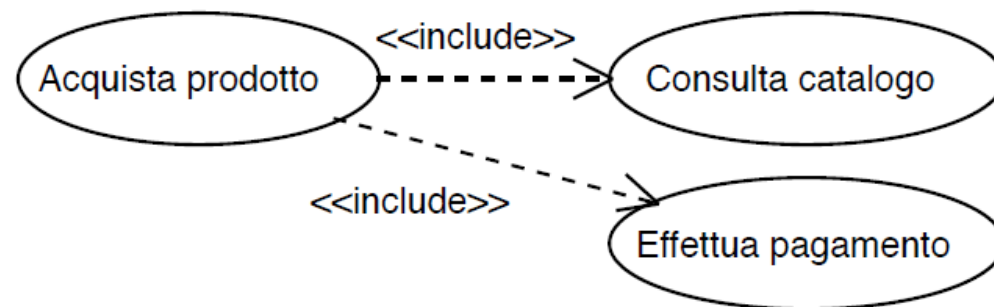
Generalizzazione tra attori

- L'attore specializzato eredita parte delle caratteristiche dell'attore generico.



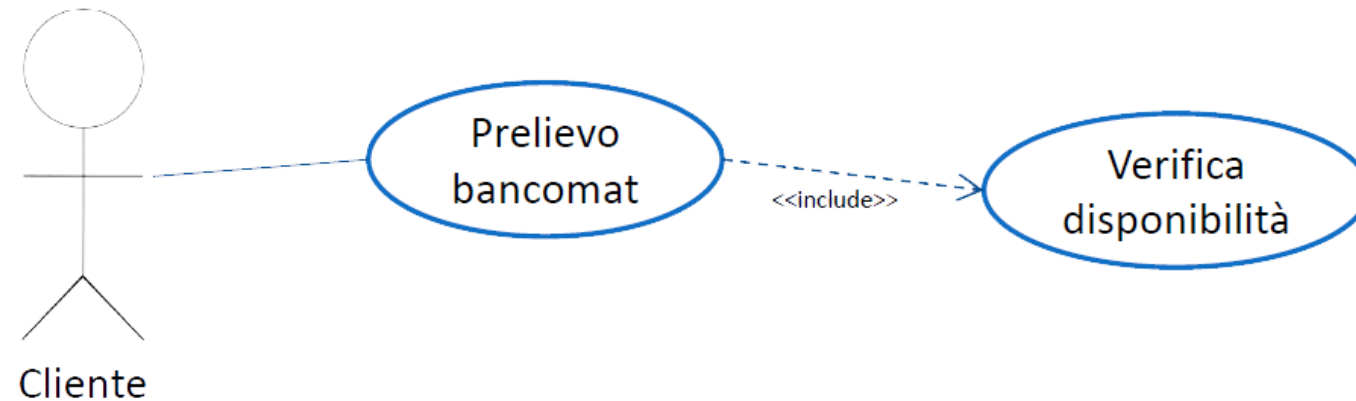
Elementi del diagramma: include

- Una dipendenza tra casi d'uso; il caso incluso fa parte del comportamento di quello che lo include.
- L'inclusione non è opzionale ed avviene in ogni istanza del caso d'uso.
- La corretta esecuzione del caso d'uso che include dipende da quella del caso d'uso incluso.
- Non si possono formare cicli di include.
- Usato per riutilizzare parti comuni a più casi d'uso.



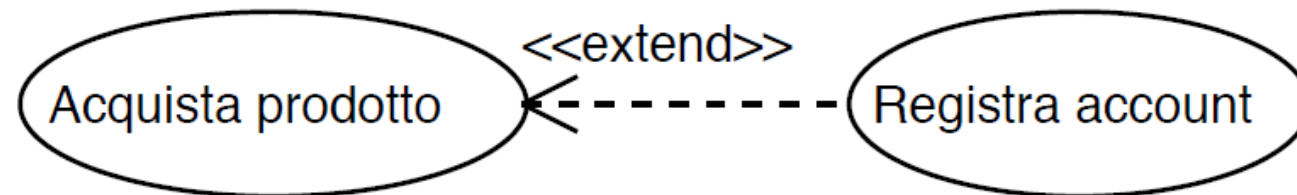
Elementi del diagramma: include

- In altri termini un caso d'uso comprende le funzionalità di un altro caso.
- Una relazione di inclusione da un caso d'uso A(base) ad un caso d'uso B(incluso) indica che le funzionalità del caso d'uso B saranno sempre presenti in un istanza del caso d'uso A.

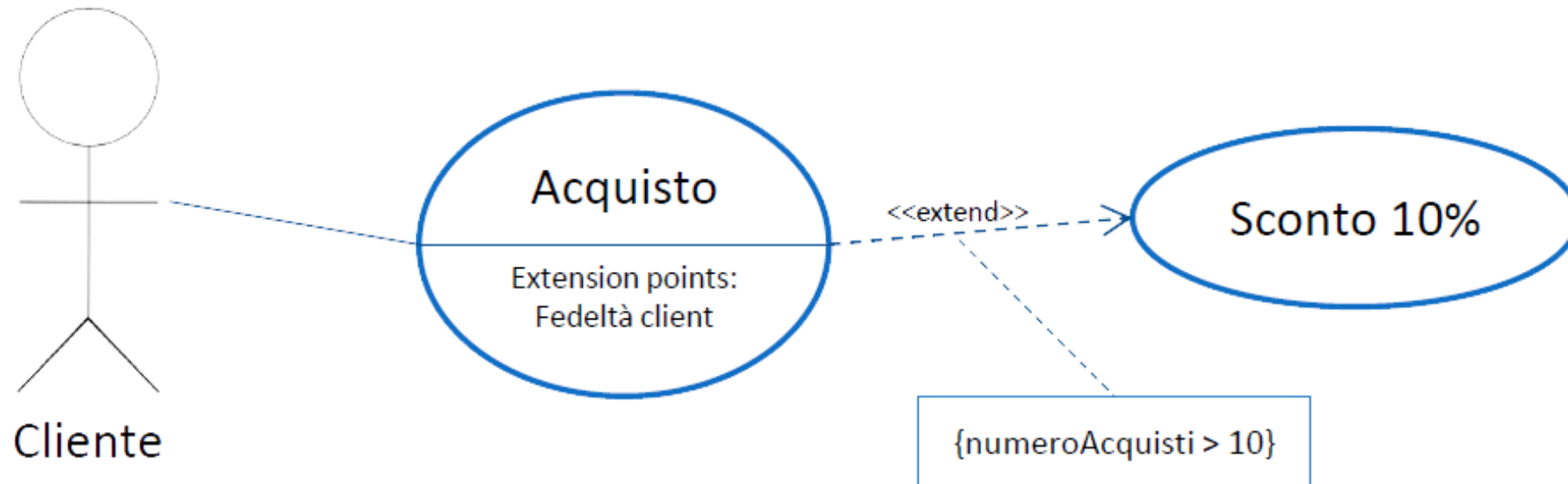


Elementi del diagramma: extend

- Una dipendenza tra casi d'uso (notare il verso della freccia).
- Il caso d'uso che estende (client) specifica un incremento di comportamento a quello esteso (supplier).
- Si tratta di comportamento supplementare ed opzionale che gestisce casi particolari o non standard.
- Diverso da una generalizzazione tra casi d'uso:
 - in una generalizzazione, entrambi i casi d'uso sono ugualmente significativi
 - in un extend, il client non ha necessariamente senso se preso da solo

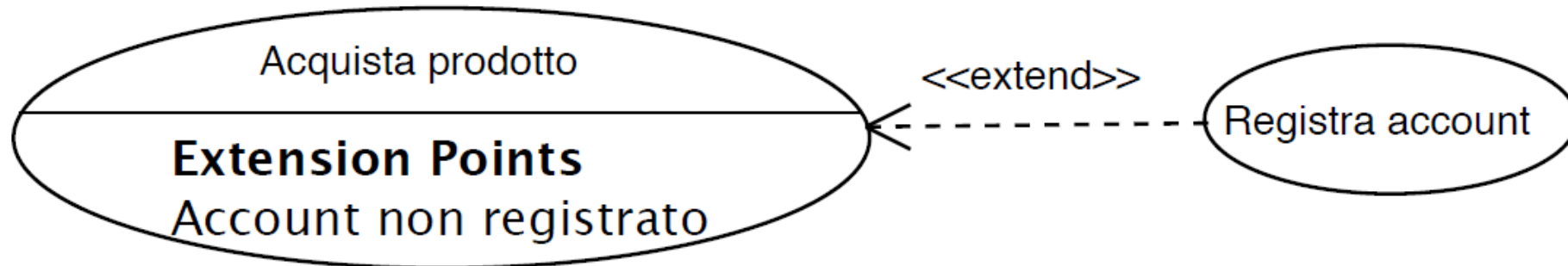


- Un caso d'uso può essere esteso nella sua funzionalità ad un altro caso
- Una relazione di estensione da un caso d'uso A ad un caso d'uso B indica che un istanza del caso d'uso B può essere maggiorata, sotto determinate condizioni specificate nell'estensione, dal comportamento di A.



Extension points

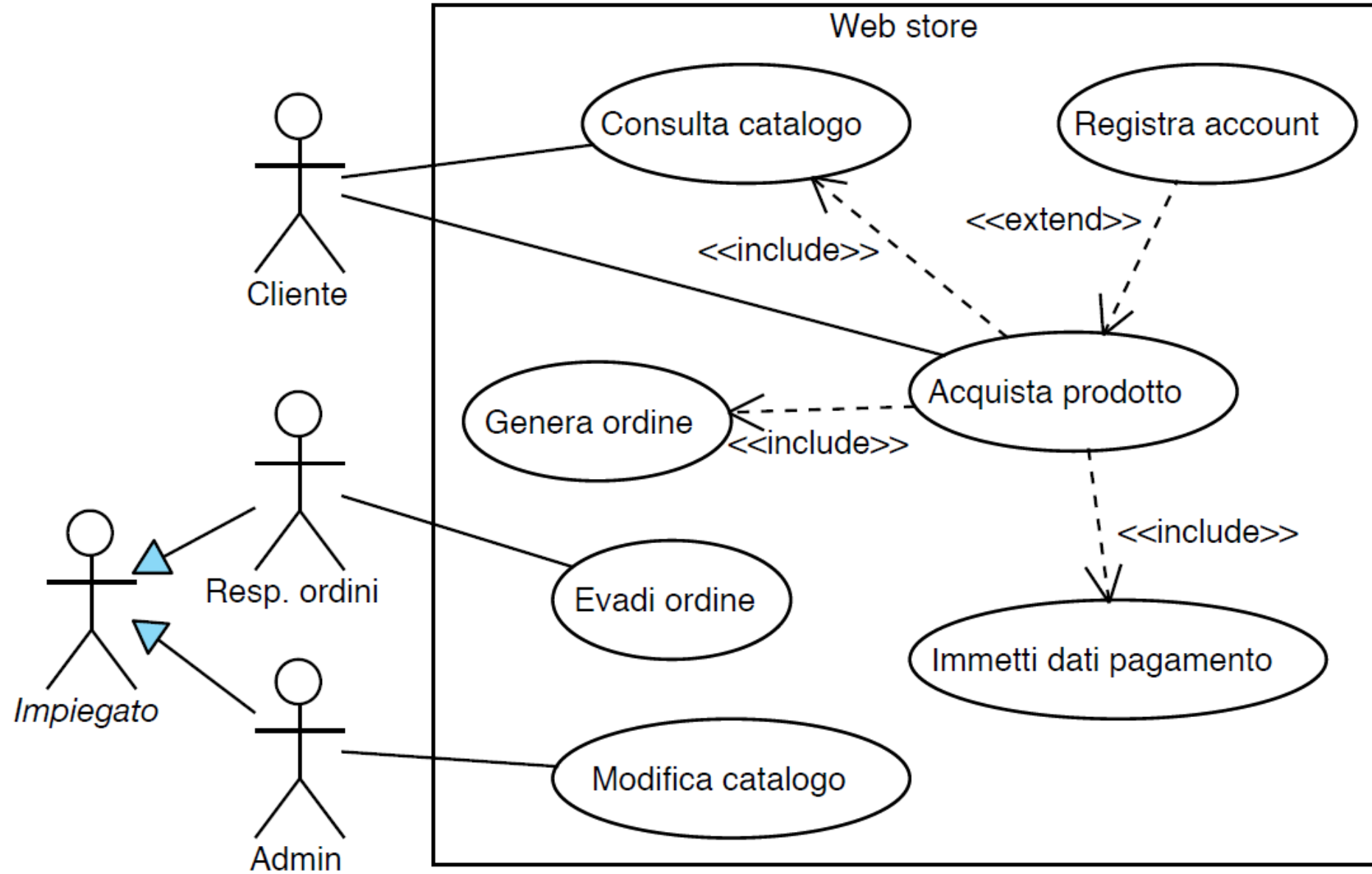
- Un caso d'uso raggiunto da almeno un extend può opzionalmente visualizzare i propri extension points.
- Specifica i punti e/o condizioni dell'esecuzione in cui il comportamento viene esteso.
- Se gli extension points sono molti, alcuni tool possono supportare la rappresentazione a rettangolo (i casi d'uso sono classificatori).



Differenze tra extend e include

- Una relazione di inclusione (A include B): indica che ogni istanza del caso d'uso A include sempre il comportamento di B
- Una relazione di estensione (A estende B): indica che al verificarsi di opportune condizioni (condizioni di guardia) il comportamento di B può essere esteso dal comportamento di A.
- Include specifica un comportamento obbligatorio mentre Extend specifica comportamento supplementare (varianti).
- Nell'include la freccia va dal caso d'uso che include verso quello incluso mentre nell'extend la freccia va dal caso d'uso che estende verso quello esteso.
- Sono entrambi costrutti utili, ma non se ne deve abusare, o la leggibilità ne risente.

Esempio diagramma



Sono sufficienti questi diagrammi?

- Spesso nasce l'esigenza di abbinare i diagrammi dei casi d'uso a specifiche testuali più formali.
- I diagrammi dei casi d'uso non sono adatti a mostrare:
 - la sequenza temporale delle operazioni
 - lo stato del sistema e degli attori prima e dopo l'esecuzione del caso d'uso
- Manca la parte più importante: la descrizione (specificata) dei casi d'uso!

TOPIC 10

Descrizione Dei Casi D'Uso

- Ogni caso d'uso costituisce una sequenza di attività che generano un risultato per l'attore che con esso interagisce.
- La sequenza di attività viene descritta in una specifica comportamentale
 - Può essere un diagramma di attività, di sequenza, di stato o una descrizione informale.
 - Generalmente si sceglie quest'ultima.

- Ogni caso d'uso ha un nome e una specifica.
- La specifica è composta da:
 - precondizioni: condizioni che devono essere vere prima che il caso d'uso si possa eseguire
 - sequenza degli eventi: i passi che compongono il caso d'uso
 - postcondizioni: condizioni che devono essere vere quando il caso d'uso termina l'esecuzione

<i>Nome del caso d'uso</i>	Caso d'uso: Pagamento IVA
<i>Identificatore univoco</i>	ID: UC1
<i>Gli attori interessati dal caso d'uso</i>	Attori: Tempo, Fisco
<i>Lo stato del sistema prima che il caso d'uso possa iniziare</i>	Precondizioni: 1. Si è concluso un trimestre fiscale
<i>I passi del caso d'uso</i>	Sequenza degli eventi: 1. Il caso d'uso inizia quando si conclude un trimestre fiscale. 2. Il sistema calcola l'ammontare dell'IVA dovuta al Fisco. 3. Il sistema trasmette un pagamento elettronico al Fisco.
<i>Lo stato del sistema quando l'esecuzione del caso d'uso è terminata</i>	Postcondizioni: 1. Il Fisco riceve l'importo IVA dovuto.

- Ogni descrizione di un caso d'uso è caratterizzata dalla descrizione di informazioni di base che permettono di specificare il contesto, chi utilizza il caso d'uso, cosa succede allo stato del sistema prima e dopo l'attivazione del caso d'uso ed uno o più scenari di interazione.
- A loro volta gli scenari di interazione si suddividono in:
 - Base, quando il caso d'uso esegue il suo compito correttamente andando opportunamente a modificare lo stato del sistema.
 - Alternativi, quando il caso d'uso termina con esiti positivi, con complicazioni o va incontro ad errori, cioè fallisce il suo compito.

Tabella descrizione Use Case

Descrizione	Note
Caso d'uso	<codice>
Data	
Nome	
Versione	<perrevisioni>
Descrizione	
Priorità	
Durata	
Punti di estensione	
Estende	
Include	
Attore primario	
Attori secondari	
Precondizioni	<condizioni necessarie per l'avvio del caso d'uso>
Postcondizioni	<condizioni da verificare per successo / insuccesso del caso d'uso>
Innesco	<cosa da avviare>
Scenario principale	
Scenario alternativo I	
Scenario alternativo II (insuccesso)	
Scenario alternativo N	
Note	
Riferimento	

- Si parte da appunti e trascrizioni di interviste e colloqui con il cliente.
- Si procede per fasi
 - Definizione attori e casi d'uso
 - Chi utilizzerà il sistema per il data entry?
 - Chi è il destinatario delle informazioni fornite dal sistema?
 - Quali altri sistemi devono interagire con questo?
- Organizzare secondo priorità i casi d'uso individuati
 - Sviluppare prima i più importanti

- Sviluppare ciascun caso d'uso
 - Creare la specifica dettagliata di ogni caso d'uso
- Strutturare il diagramma dei casi d'uso
 - Aggiunta di attori e casi d'uso
 - Generalizzazioni
 - Inclusioni
 - Estensioni
 - Raggruppamento in unità logiche (package)

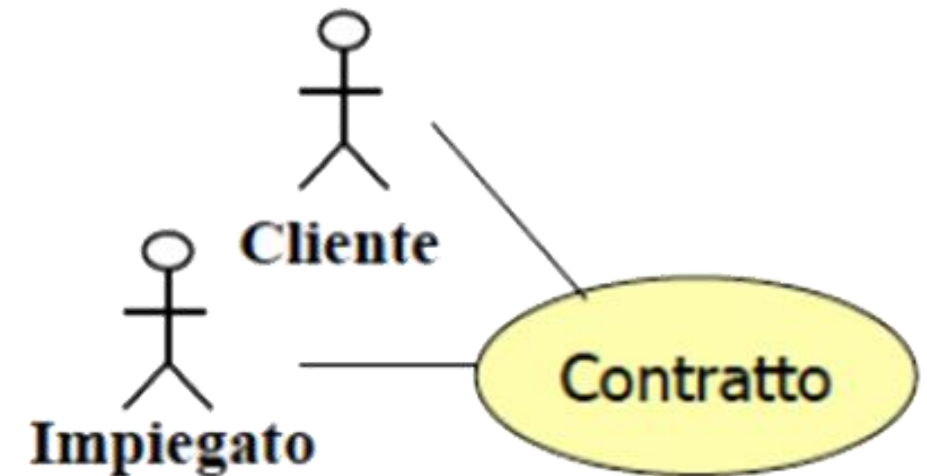
- Un elenco di azioni che definisce il caso d'uso nella sua completezza.
- Il caso d'uso si considera eseguito solo se l'esecuzione arriva fino alla fine.
- Un'azione è sempre iniziata da un attore oppure dal sistema (in UML, gli eventi sono sempre legati a chi li crea).
 - Passo iniziale: 1. Il caso d'uso inizia quando <attore> <azione>...
 - Passi successivi: <numero>. Il <attore/sistema> <azione>

Sequenza (corretta) di eventi

- ~~• Incomincia quando si seleziona la funzione «ordina libro»~~
- Il caso d'uso inizia quando il cliente seleziona la funzione «ordina libro»
- ~~• Vengono inseriti i dati del cliente~~
- Il cliente inserisce nel form il suo nome e indirizzo
- Il sistema verifica i dati del Cliente

Un altro esempio di sequenza di eventi

- Nome: Contratto
- Precondizione: l'impiegato è connesso
- Flusso principale degli eventi
 - L'impiegato inserisce il nome cliente e numero di conto
 - Controlla la loro validità
 - Inserisce il numero di azioni da comprare e ID azienda quotata
 - Il sistema determina il prezzo
 - Il sistema controlla il limite
 - Il sistema manda l'ordine in Borsa
 - L'impiegato memorizza il numero di conferm



- Ogni caso d'uso:
 - Ha una sequenza di transizioni (eventi) normale o di base
 - Può avere varie sequenze alternative
 - Ha varie sequenze di transazioni eccezionali per la gestione di errori o casi particolari
- Per descrivere correttamente il flusso di eventi quindi:
 - Descrive solo gli eventi relativi al caso d'uso, e non quel che avviene in altri casi d'uso
 - Descrivere come e quando il caso d'uso inizia e finisce
 - Descrivere il flusso base degli eventi
 - Descrivere ogni flusso alternativo

- UML usa parole chiave per esprimere queste variazioni alla sequenza principale.
- Parola chiave Se: indica una ramificazione della sequenza degli eventi nel caso si verifichi una condizione.
- Ripetizioni all'interno di una sequenza:
 - parola chiave Per (For)
 - parola chiave Fintantoché (While)
- È bene non eccedere con le ramificazioni!

- Facciamo un esempio partendo dal caso d'uso «aggiorna carrello» di un negozio on-line.
- Possibili ramificazioni, dopo aver aggiunto un articolo al carrello:
 - se il cliente richiede una nuova quantità il sistema aggiorna la quantità di quell'articolo
 - se il client seleziona rimuovi articolo il sistema elimina quell'articolo dal carrello
- Le sequenze alternative sono invece ramificazioni che non possono essere espresse utilizzando il Se.
 - Ad esempio dovute a condizioni che si possono verificare in un qualunque momento: il cliente abbandona la pagina del carrello

Caso d'uso: AggiornaCarrello
ID: UC2
Attori: Cliente
Precondizioni: 1. Il contenuto del carrello è visibile
Sequenza degli eventi: 1. Il caso d'uso inizia quando il Cliente seleziona un articolo nel carrello. 2. Se il Cliente seleziona “rimuovi articolo” 2.1 Il Sistema elimina l'articolo dal carrello. 3. Se il Cliente digita una nuova quantità 3.1 Il Sistema aggiorna la quantità dell'articolo presente nel carrello
Postcondizioni: 1. Il contenuto del carrello è stato aggiornato
Sequenza alternativa 1: 1. In qualunque momento il Cliente può abbandonare la pagina del carrello
Postcondizioni:

Come individuare sequenze alternative?

- Ad ogni passo della sequenza degli eventi principale, cercare:
 - alternative all'azione eseguita in quel passo
 - errori possibili nella sequenza principale
 - interruzioni che possono avvenire in qualunque momento della sequenza principale
- Sequenze alternative abbastanza complesse possono essere descritte separatamente.
 - stessa sintassi, si sostituisce «caso d'uso» con «sequenza degli eventi alternativa»
 - il primo passo può indicare il punto della sequenza principale da cui si proviene

Apertura Conto Corrente

1. il cliente si presenta in banca per aprire un nuovo c/c
2. l'addetto riceve il cliente e fornisce spiegazioni
3. se il cliente accetta fornisce i propri dati
4. l'addetto verifica se il cliente è censito in anagrafica
5. l'addetto crea il nuovo conto corrente
6. l'addetto segnala il numero di conto al cliente

Varianti

1. (a) se il cliente non accetta il caso d'uso termina
2. (b) se il conto va intestato a più persone vanno forniti i dati di tutte
3. (a) se il cliente (o uno dei diversi intestatari) non è censito l'addetto provvede a registrarlo

Il punto 5 (crea conto) può essere ulteriormente dettagliato:

1. l'addetto richiede al sistema la transazione di inserimento nuovo conto
2. il sistema richiede i codici degli intestatari
3. l'addetto fornisce i codici al sistema
4. il sistema fornisce le anagrafiche corrispondenti, e richiede le condizioni da applicare al conto
5. l'addetto specifica le condizioni e chiede l'inserimento
6. il sistema stampa il contratto con il numero del conto

Varianti:

1. (a) se il sistema non riconosce il cliente, o se fornisce un'anagrafica imprevista, l'addetto può effettuare correzioni o terminare l'inserimento Ingegneria

- Uno scenario rappresenta una particolare interazione tra uno o più attori e il sistema.
- Non contiene ramificazioni o sequenze alternative: è semplicemente la cronaca di un'interazione vera o verosimile.
 - il concetto risulta più immediato pensando agli attori in maniera concreta: non un generico cliente, ma Alice o Bob
- Una definizione alternativa: un caso d'uso è un insieme di scenari possibili se un utente prova a raggiungere un dato risultato

Scenario principale e scenari secondari

- Corrispondono ad esecuzioni della sequenza principale e di quelle alternative, rispettivamente.
- Strategie per limitare il numero, potenzialmente enorme, degli scenari secondari:
 - documentare solo quelli considerati più importanti
 - se ci sono scenari secondari molto simili, se ne documenta uno solo, se necessario aggiungendo annotazioni per spiegare come gli altri scenari differiscano dall'esempio.

- Diagrammi di flusso invece di casi d'uso: un caso d'uso è una sequenza di azioni, non una singola azione!
- Nome del caso d'uso che appare più volte nel diagramma
- Le frecce tra i casi d'uso non sono tratteggiate (- - - - - ->) o etichettate «extend» o «include»
- «extend»: la freccia va dal caso che descrive l'evento alternativo al caso standard
- «include»: la freccia va dal caso chiamante al caso che descrive le azioni da includere

- Assenza di precondizioni
- Mancata connessione alla rappresentazione grafica
- Nomi diversi per le stesse entità nelle rappresentazioni grafica e testuale
- Flusso eccezionale: mancanza di indicazioni nel flusso principale del punto in cui va controllata la condizione eccezionale

- Analizzare il materiale contenente i requisiti
- Creare un glossario di progetto con parole chiave e una breve descrizione
- Capire chi sono gli attori
- Estrarre i casi d'uso più evidenti
- Cominciare ad organizzare i casi d'uso in un diagramma
- Modellare i casi d'uso come sequenze di eventi
- Raffinare progressivamente se necessario

- Il diagramma UML dei casi d'uso è un tool per la modellazione del comportamento di un sistema.
- Descrive gli attori che interagiscono con il sistema, cosa fanno, e cosa ottengono dal sistema.
- A questo punto non interessa sapere come il sistema fornisca il comportamento richiesto.